

SWI-Prolog 中文文档

目录

1. 快速入门	3
2. 用户的初始化文件	6
3. 初始化文件和目标	7
4. 命令行选项	8
5. UI 主题	13
6. GNU Emacs 接口	14
7. 在线帮助	15
8. 命令行历史	18
9. 复用顶层绑定	19
10. 调试器概览	20
11. 加载和运行项目	32
12. 环境控制 (Prolog 标志)	37
13. 钩子谓词概览	60
14. 库的自动加载	62
15. SWI-Prolog 语法	64
16. 有理树 (循环项)	72
17. 即时子句索引	73
18. 宽字符支持	76
19. 系统限制	78
20. SWI-Prolog 和 32 位机器	80
21. 二进制兼容性	81

快速入门

1.1 启动 SWI-Prolog

1.1.1 在 Unix 上启动 SWI-Prolog

默认情况下，SWI-Prolog 安装后的可执行程序名为 `swipl`。SWI-Prolog 本身及其实用程序的命令行参数使用标准 Unix `man` 手册页记录。SWI-Prolog 通常作为交互式应用程序运行，直接启动程序即可：

```
$ swipl
Welcome to SWI-Prolog ...
...

1 ?-
```

启动 Prolog 后，通常使用 `consult/1` 将程序载入系统。也可以把程序文件名放在方括号中作为简写。下面的目标会载入文件 `likes.pl`，其中包含谓词 `likes/2` 的子句：

```
?- [likes].
true.

?-
```

也可以把源文件作为命令行参数传给 `swipl`：

```
$ swipl likes.pl
Welcome to SWI-Prolog ...
...

1 ?-
```

上面两个例子都假设 `likes.pl` 位于当前工作目录。如果使用命令行版本 `swipl`，工作目录就是启动 SWI-Prolog 的 shell 所在目录。如果启动的是 GUI 版本 `swipl-win`，工作目录则主要取决于操作系统。可以使用 `pwd/0` 和 `cd/0` 查看和修改工作目录。实用程序 `ls/0` 会列出工作目录内容。

```
?- pwd.
% /home/janw/src/swipl-devel/linux/
true.
?- cd('~'/tmp').
true.

?- pwd.
% /home/janw/tmp/
true.
```

文件 `likes.pl` 也安装在 SWI-Prolog 安装目录的 `demo` 子目录中，因此可以不依赖当前工作目录，用下面的命令载入。关于 SWI-Prolog 如何指定文件位置，请参阅 [absolute_file_name/3](#) 和 [file_search_path/2](#)。

```
?- [swi(demo/likes)].
true.
```

从这里开始，Unix 和 Windows 用户面对的内容相同。如果你使用 Unix，可以继续阅读“从控制台添加规则”。

1.1.2 在 Windows 上启动 SWI-Prolog

在 Windows 系统上安装 SWI-Prolog 后，用户会得到以下重要内容：

- 一个名为 `swipl` 的文件夹，本文档后续称为目录；其中包含系统的可执行文件、库等。不会在该目录之外安装文件。
- 程序 `swipl-win.exe`，提供一个用于与 Prolog 交互的窗口。程序 `swipl.exe` 是运行在控制台窗口中的 SWI-Prolog 版本。
- 文件扩展名 `.pl` 会关联到程序 `swipl-win.exe`。打开 `.pl` 文件会启动 `swipl-win.exe`，切换到被打开文件所在目录，并载入该文件。

启动前文提到的 `likes.pl` 文件，通常只需在 Windows 资源管理器中双击该文件。

1.2 从控制台添加规则

虽然强烈建议把程序放在文件中，并在需要时编辑文件再使用 `make/0` 重新载入，但也可以直接从终端管理事实和规则。添加少量子句最方便的方法是 `consult` 伪文件 `user`。输入用系统的文件结束字符结束。

```
?- [user].
|: hello :- format('Hello world~n').
|: ^D
true.

?- hello.
Hello world
true.
```

谓词 `assertz/1` 和 `retract/1` 也可以用来添加和删除规则或事实。

1.3 执行查询

载入程序后，就可以向 Prolog 询问关于该程序的问题。下面的查询询问 Prolog：sam 喜欢什么食物。如果系统可以证明某个 `x` 满足该目标，它会以 `x = <value>` 的形式回答。若用户想要另一个解，可以输入分号；或空格键。若不想查看更多答案，按回车即可。如果用户按回车，或者 Prolog 知道已经没有更多答案，Prolog 会用句号 `.` 结束输出。如果 Prolog 找不到更多答案，会写出 `false.`。最后，如果查询或程序包含错误，Prolog 会用错误消息回答。

```
?- likes(sam, X).
X = dahl ;
```

```
X = tandoori ;
...
X = chips.

?-
```

注意，Prolog 写出的答案本身也是有效的 Prolog 程序；执行它可以得到与原程序相同的一组答案。

1.4 检查和修改程序

如果配置正确，谓词 `edit/1` 会根据参数启动内置编辑器或用户配置的编辑器。参数可以是任何能够关联到位置的对象：文件名、谓词名、模块名等。如果参数只解析到一个位置，编辑器会在该位置打开；否则系统会让用户选择。

如果存在图形用户界面，编辑器通常会创建新窗口，而系统会继续提示下一个命令。用户可以编辑源文件、保存文件，然后运行 `make/0` 来更新所有修改过的源文件。如果编辑器无法在窗口中打开，它会在同一控制台中打开；退出编辑器时会运行 `make/0`，重新载入任何已修改的源文件。

```
?- edit(likes).

true.
?- make.
% /home/jan/src/pl-devel/linux/likes compiled 0.00 sec, 0 clauses

?- likes(sam, X).
...

```

也可以使用 `listing/1` 反编译程序，如下所示。`listing/1` 的参数可以只是谓词名，也可以是形如 `Name/Arity` 的谓词指示器，例如 `?- listing(mild/1).`；还可以是一个头，例如 `?- listing(likes(sam, _)).`，用于列出所有匹配的子句。不带参数的 `listing/0` 会列出整个程序。

```
?- listing(mild).
mild(dahl).
mild(tandoori).
mild(kurma).

true.
```

1.5 停止 Prolog

交互式顶层可以通过两种方式停止：输入系统文件结束字符，通常是 `Control-D`；或者执行谓词 `halt/0`：

```
?- halt.
$
```

用户的初始化文件

系统初始化完成后，会 consult 用户的初始化文件。该文件通过 `absolute_file_name/3` 查找，并使用路径别名 `app_config`。 `app_config` 指向操作系统默认的应用配置数据目录下名为 `swi-prolog` 的目录：

- 在 Windows 上，它是 CSIDL 文件夹 `CSIDL_APPDATA`，通常类似 `C:\Documents and Settings\username\Application Data`。
- 如果设置了环境变量 `XDG_DATA_HOME`，则使用该变量。这遵循 [freedesktop](#) 标准。
- 否则使用 `~/.config` 的展开结果。

可以用下面的调用找到该目录：

```
?- absolute_file_name(app_config(.), Dir, [file_type(directory)]).  
Dir = '/home/jan/.config/swi-prolog'.
```

找到第一个启动文件后，系统会载入它，并停止继续寻找其他启动文件。启动文件名可以用 `-f file` 选项修改。如果 `File` 是绝对路径，则直接载入该文件；否则使用与默认启动文件相同的约定查找。最后，如果 `file` 为 `none`，则不载入任何文件。

安装目录提供了 `customize/init.pl` 文件，其中包含一些常用于定制 Prolog 行为的命令，这些命令默认被注释掉，例如编辑器集成、颜色选择或历史记录参数。许多开发工具也提供菜单项，用于编辑启动文件，或从系统骨架创建新的启动文件。

另请参阅命令行选项中的 `-s`（脚本）和 `-F`（系统范围初始化），以及“初始化文件和目标”。

初始化文件和目标

通过命令行参数，SWI-Prolog 可以被强制载入文件，并执行用于初始化或非交互式运行的查询。最常用的选项包括：用 `-f file` 或 `-s file` 让 Prolog 载入文件，用 `-g goal` 定义初始化目标，用 `-t goal` 定义顶层目标。下面是一个从命令行直接启动应用程序的典型例子：

```
machine% swipl -s load.pl -g go -t halt
```

它告诉 SWI-Prolog：载入 `load.pl`，使用入口点 `go/0` 启动应用程序，并在 `go/0` 完成后退出，而不是进入交互式顶层。

命令行中可以多次出现 `-g goal`。这些目标会按顺序执行。每个目标留下的可能选择点都会被剪除。如果某个目标失败，执行会以退出状态 1 停止。如果某个目标引发异常，系统会打印异常，并以退出代码 2 停止进程。

可以使用 `-q` 抑制所有信息性消息，也会抑制初始化目标失败时通常打印的错误消息。

在 MS-Windows 中，也可以用定义了适当命令行参数的快捷方式实现同样效果。另一种常见做法是编写一个 `run.pl` 文件，内容如下。双击 `run.pl` 即可启动应用程序。

```
:- [load].           % load program
:- go.               % run it
:- halt.             % and exit
```

“使用 PrologScript”会讨论更多脚本选项；运行时章节会讨论运行时可执行文件的生成。运行时可执行文件是一种交付可执行程序的方式，运行它时不需要完整的 Prolog 系统。

命令行选项

SWI-Prolog 可以以下列模式之一执行：

- `swipl --help`
- `swipl --version`
- `swipl --arch`
- `swipl --dump-runtime-variables`

这些选项必须作为唯一选项出现。它们会让 Prolog 打印信息性消息并退出。参见“信息性命令行选项”。

- `swipl [option ...] script-file [arg ...]`

在 Unix 系统中，如果执行的文件以 `#!/path/to/executable [option ...]` 开头，则会使用这种参数形式。脚本文件之后的参数可以通过 Prolog 标志 `argv` 访问。

- `swipl [option ...] prolog-file ... [--] arg ...`

这是启动 Prolog 的常规方式。选项见“运行 Prolog 的命令行选项”、“控制堆栈大小”和“从命令行运行目标”。Prolog 标志 `argv` 提供对 `arg ...` 的访问。如果选项后跟一个或多个 Prolog 文件名，也就是扩展名为 `.pl`、`.prolog`，或在 Windows 上安装期间注册的用户首选扩展名的文件，则这些文件会被载入。第一个文件会记录在 Prolog 标志 `associated_file` 中。此外，`pl-win[.exe]` 会使用 `working_directory/2` 切换到这个主源文件所在的目录。

- `swipl -o output -c prolog-file ...`

选项 `-c` 用于把一组 Prolog 文件编译成可执行文件。参见“编译选项”。

- `swipl -o output -b bootfile prolog-file ...`

引导编译。参见“维护选项”。

4.1 信息性命令行选项

--arch

作为唯一选项给出时，打印体系结构标识符并退出。该标识符也可通过 Prolog 标志 `arch` 查看。另请参阅 `--dump-runtime-variables`。

--dump-runtime-variables[=format]

作为唯一选项给出时，打印一组变量设置，可在 shell 脚本中用于处理 Prolog 参数。`swipl-ld` 也使用这个功能。下面是一个典型用法：

```
eval `swipl --dump-runtime-variables`
cc -I$PLBASE/include -L$PLBASE/lib/$PLARCH ...
```

该选项可以跟 `=sh`，以 POSIX shell 格式输出；这是默认值。也可以跟 `=cmd`，以兼容 MS-Windows `cmd.exe` 的格式输出。

--help

作为唯一选项给出时，概述最重要的选项。

--version

作为唯一选项给出时，概述版本和体系结构标识符。

--abi-version

打印一个表示多方面二进制兼容性的键，也就是字符串。参见“二进制兼容性”。

4.2 运行 Prolog 的命令行选项

注意，布尔选项可以写成 `--name` 表示 `true`，也可以写成 `--noname` 或 `--no-name` 表示 `false`。下文采用 `--no-name` 的形式，因为默认值为 `true`。

-D name[=value]

将 Prolog 标志 `name` 设置为 `value`。这些标志会在载入初始保存状态后立即设置。如果标志已经定义，`value` 会被转换为该标志的类型。如果标志尚未定义，且 `value` 表示数字，则将其设置为数字，否则设置为原子。如果没有给出 `=value`，则使用布尔值。如果 `name` 形如 `no-flag`，则把 `flag` 设置为 `false`；否则把名为 `name` 的标志设置为 `true`。`name[=value]` 可以紧跟在 `-D` 后面，也可以作为下一个命令行参数出现。

许多命令行选项也会反映为 Prolog 标志。系统意图把这些形式作为同义项处理。目前，一些命令行标志会在保存状态载入完成前影响 Prolog 初始化，而另一些标志在 Prolog 初始化后可能无法更改。例如，未来版本将支持 `-Dhome=dir`，用于修改 Prolog 安装目录的概念。

--debug-on-interrupt

立即启用在中断信号上调试，也就是 `Control-C` 或 `SIGINT`。通常，中断调试会在进入交互式顶层时启用。该标志可用于在执行来自 `-g` 或 `initialization/[1,2]` 的目标时，通过中断启动调试器。另请参阅 Prolog 标志 `debug_on_interrupt`。

--home[=DIR]

如果带 `DIR`，则把 SWI-Prolog 的 `home` 目录设置为 `DIR`，并向进程添加环境变量 `SWI_HOME_DIR`，其值为 `DIR`。如果不带参数，则报告找到的 `home` 目录并退出；如果找不到位置，则打印错误并以状态 1 退出。关于 `home` 目录的定位方式，参见相关章节；关于 SWI-Prolog 如何使用该目录，参见 Prolog 标志 `home`。

--quiet

将 Prolog 标志 `verbose` 设置为 `silent`，抑制信息性消息和横幅消息。也可以写作 `-q`。

--no-debug

禁用调试。详情参见 `current_prolog_flag/2` 标志 `generate_debug_info`。

--no-signals

禁止 Prolog 处理任何信号。这一属性有时适合嵌入式应用程序。该选项会把标志 `signals` 设置为 `false`。注意，用于解除系统调用阻塞的处理器仍会安装。可以额外使用 `--sigalert=0` 阻止这一点。另请参阅 `--sigalert`。

--no-threads

在运行时禁用多线程版本的线程功能。另请参阅标志 `threads` 和 `gc_thread`。

--no-packs

不附加扩展包，也就是 `add-ons`。另请参阅 `attach_packs/0` 和 Prolog 标志 `packs`。

--no-pce

启用或禁用 `xpce` GUI 子系统。默认情况下，如果已安装 `xpce` 且系统可以访问图形环境，它会作为可自动载入组件提供。使用 `--pce` 会在用户空间中载入 `xpce` 系统；使用 `--no-pce` 会使其在本会话中不可用。

--on-error=style

指定如何处理错误。详情参见 Prolog 标志 `on_error`。

--on-warning=style

指定如何处理警告。详情参见 Prolog 标志 `on_warning`。

--pldoc[=port]

在一个空闲网络端口启动 PDoc 文档系统，并在 `http://localhost:port` 打开用户浏览器。如果指定了 `port`，服务器会在给定端口启动，并且不会启动浏览器。

--sigalert=NUM

使用信号 `NUM`，范围为 1 到 31，提醒线程。这是为了让 `thread_signal/2` 以及派生的 Prolog 信号处理在目标线程阻塞于可中断系统调用时立即生效，例如 `sleep/1` 或对多数设备的读写。默认使用 `SIGUSR2`。如果 `NUM` 为 0，则不安装该处理器。可以用 `prolog_alert_signal/2` 在运行时查询或修改该值。

--no-tty

仅限 Unix。该开关控制终端，以允许向跟踪器和 `get_single_char/1` 发送单字符命令。默认情况下，除非系统检测到自己没有连接到终端，或正作为 GNU Emacs 下级进程运行，否则会启用终端操作。另请参阅 Prolog 标志 `tty_control`。

--win-app

该选项只在 `swipl-win.exe` 中可用，用于开始菜单项。它会让 `plwin` 在 `...\My Documents\Prolog` 或其本地等效文件夹中启动。若 Prolog 子目录不存在，则创建它。另请参阅 `win_folder/2`。

-O

优化编译。详情参见 `current_prolog_flag/2` 标志 `optimise`。

-l file

载入 `file`。该标志提供与其他一些 Prolog 系统的兼容性。在 SWI-Prolog 中，它用于跳过通过 `initialization/2` 指令指定的程序初始化。另请参阅脚本相关章节和 `initialize/0`。

-s file

将 `file` 用作脚本文件。脚本文件会在通过 `-f file` 选项指定的初始化文件之后载入。不同于 `-f file`，使用 `-s` 不会阻止 Prolog 载入个人初始化文件。

-f file

使用 `file` 作为初始化文件，而不是默认的 `init.pl`。`-f none` 会停止 SWI-Prolog 搜索启动文件。该选项可以替代 `-s file`，用于阻止 Prolog 载入个人初始化文件。另请参阅“用户的初始化文件”。

-F script

从 SWI-Prolog home 目录选择启动脚本。脚本文件名为 `<script>.rc`。默认的 `script` 名称从可执行文件推导而来，取程序名开头的字母数字字符，包括字母、数字和下划线。`-F none` 会停止查找脚本。该机制用于简单管理稍有差异的版本。例如，可以编写脚本 `iso.rc`，再使用 `pl -F iso` 选择 ISO 兼容模式；或者从 `iso-pl` 到 `pl` 建立链接。

-x bootfile

从 `bootfile` 启动，而不是从系统默认引导文件启动。引导文件可以是使用 `-b` 或 `-c` 选项进行 Prolog 编译得到的文件，也可以是使用 `qsave_program/[1,2]` 保存的程序。

-p alias=path1[:path2 ...]

为 `file_search_path` 定义路径别名。alias 是别名名称，path1 ... 是该别名的值列表。在 Windows 上，列表分隔符是 `;`；在其他系统上是 `:`。值可以是形如 `alias(value)` 的项，也可以是路径名。计算出的别名会使用 `asserta/1` 添加到 `file_search_path/2`，因此它们排在该别名的预定义值之前。关于这种文件定位机制的细节，参见 `file_search_path/2`。

--traditional

该标志禁用 SWI-Prolog 7 中引入的最重要扩展，这些扩展会造成与早期版本的不兼容。具体来说，列表会以传统方式表示，双引号文本表示为字符代码列表，并且不支持字典上的函数记法。如果存在该标志，字典作为语法实体以及作用于字典的谓词仍然可用。

--

停止扫描更多参数。这样可以在它之后向应用程序传递参数。可以通过 `current_prolog_flag/2` 读取标志 `argv`，以获得命令行参数。

4.3 控制堆栈大小

从 7.7.14 版本起，堆栈不再分别受限，而只限制组合大小。注意，32 位系统仍然有 128Mb 限制。默认情况下，64 位机器上的组合限制为 1Gb，32 位机器上为 512Mb。

例如，要把堆栈限制为 32Gb，可以使用下面的命令。注意，堆栈限制按线程应用。单个线程可以通过 `thread_create` 的 `stack_limit(+Bytes)` 选项控制。任意线程都可以调用 `set_prolog_flag(stack_limit, Limit)` 来调整堆栈限制。该限制会被从此线程创建的线程继承。

```
$ swipl --stack-limit=32g
```

--stack-limit=size[bkmg]

将 Prolog 堆栈的组合大小限制为指定的 `size`。后缀把值指定为字节、Kbytes、Mbytes 或 Gbytes。

--table-space=size[bkmg]

限制表空间。表空间用于保存记忆化答案的 `trie`，这些答案来自制表。默认值在 64 位机器上为 1Gb，在 32 位机器上为 512Mb。另请参阅 Prolog 标志 `table_space`。

--shared-table-space=size[bkmg]

限制共享表使用的表空间。参见共享制表相关章节。

4.4 从命令行运行目标

-g goal

Goal 会在进入顶层之前执行。该选项可以出现多次。详情见“初始化文件和目标”。如果没有初始化目标，系统会调用 `version/0` 打印欢迎消息。可以用 `--quiet` 抑制欢迎消息，也可以用 `-g true` 达到同样效果。goal 可以是复杂项。在这种情况下，通常需要使用引号，避免被 shell 展开。下面是非交互式运行目标的一种安全方式。如果 `go/0` 成功，`-g halt` 会让进程以退出代码 0 停止；如果失败，退出代码为 1；如果引发异常，退出代码为 2。

```
% swipl <options> -g go -g halt
```

-t goal

使用 goal 作为交互式顶层，而不是默认目标 prolog/0。goal 可以是复杂项。如果顶层目标成功，SWI-Prolog 以状态 0 退出。如果失败，退出状态为 1。如果顶层引发异常，该异常会作为未捕获错误打印，并且顶层会重新启动。该标志也决定 break/0 和 abort/0 启动的目标。如果想阻止用户进入交互模式，可以用 -g goal -t halt 启动应用程序。

4.5 编译选项

-c file ...

把文件编译为“中间代码文件”。参见“加载和运行项目”。

-o output

与 -c 或 -b 结合使用，用于确定编译输出文件。

4.6 维护选项

下列选项用于系统维护，仅作为参考列出。

-b initfile ... -c file ...

引导编译。initfile ... 由 C 编写的 bootstrap 编译器编译，file ... 由普通 Prolog 编译器编译。仅用于系统维护。

-d token1,token2,...

为带有指定 token 的 DEBUG 语句打印调试消息。只有在系统以 -D0_DEBUG 标志编译时才有效。仅用于系统维护。

UI 主题

UI（颜色）主题在两个部分发挥作用：写入控制台时，以及用于基于 `xpce` 的开发工具时，例如 `PceEmacs` 或图形调试器。彩色控制台输出基于 `ansi_format/3`。基于 `print_message/2` 的中央消息基础设施会用一个 Prolog 项标记消息或消息组件，以说明其角色。钩子 `prolog:console_color/2` 会把这些角色映射到具体颜色。IDE 主题则使用 `xpce` 类变量；这些变量会在载入 `xpce` 时由 Prolog 初始化。

主题实现为 Prolog 文件，位于文件搜索路径 `library/theme` 中。可以在用户初始化文件中使用下面这样的指令载入主题：

```
:- use_module(library(theme/dark)).
```

系统提供了主题文件 `library(theme/auto)`，用于根据环境自动选择一个合理的主题。当前版本会在兼容 `xterm` 的终端模拟器上检测背景色；这类终端常见于多数 Unix 系统。如果背景色偏暗，则载入 `dark` 主题。

下面是 SWI-Prolog 支持平台上的一些注意事项：

- Unix/Linux

如果使用兼容 `xterm` 的终端模拟器运行 Prolog，可能需要显式载入某个主题，或载入 `library(theme/auto)`。

- Epilog Prolog 控制台

图形应用程序 `swipl-win` 可以通过载入主题文件来设置主题。主题文件也会设置 Epilog 控制台的前景色和背景色。

5.1 主题支持状态

主题支持是在 SWI-Prolog 8.1.11 中加入的。目前只覆盖了一部分 IDE 工具，唯一额外提供的主题 `dark` 也还没有很好地平衡。主题文件与 IDE 组件之间的接口，尤其是相关组件接口，还没有很好地稳定下来。欢迎通过改进 `dark` 主题做出贡献。等它完整并正常工作后，就可以开始添加新的主题。

GNU Emacs 接口

SWI-Prolog 通过 `sweep` 包与 GNU Emacs 紧密集成。该包把 SWI-Prolog 嵌入为一个动态 Emacs 模块，使得 Prolog 查询可以直接从 Emacs Lisp 执行。配套的 Emacs 包 `sweepprolog.el` 可以通过标准 Emacs 包管理器 `package.el` 安装；它构建在这种嵌入能力之上，为 GNU Emacs 中的 SWI-Prolog 提供完整集成的开发环境。

GNU Emacs 默认附带一个名为 `prolog.el` 的 Prolog 模式。与 `sweepprolog.el` 相比，该模式因缺少合适的 Prolog 解析器而存在一些问题。Masanobu Umeda 编写的原始 `prolog.el` 自 1989 年起就包含在 GNU Emacs 中；2006 年，Stefan Monnier 为 `prolog.el` 添加了对 SWI-Prolog 的显式支持。2011 年，原始实现的大部分被新的 Prolog 模式替换；该模式最初由 Stefan Bruda 为 XEmacs 移植编写，后来由 Stefan Monnier 改编到 GNU Emacs。此后，Stefan Monnier 与其他 GNU Emacs 贡献者一起维护它。该模式的用户可以在 <https://www.metalevel.at/pceprolog/> 找到有用的配置建议。

其他可用于配合 SWI-Prolog 工作的 Emacs 包包括：

- <https://www.metalevel.at/ediprolog/>

直接在 Emacs buffer 中与 SWI-Prolog 交互。

- <https://www.metalevel.at/etrace/>

使用 Emacs 跟踪 Prolog 代码。

- <https://emacs-lsp.github.io/dap-mode/page/configuration/#swi-prolog>

通过 `dap-mode` 以及 https://github.com/eshelyaron/debug_adapter 中的 `debug_adapter` 包，在 Emacs 中为 SWI-Prolog 提供调试适配器协议（DAP）支持。

- <https://emacs-lsp.github.io/lsp-mode/page/lsp-prolog/>

通过 `lsp-mode` 以及 https://github.com/jamesnvc/lsp_server 中的 `lsp_server` 包，在 Emacs 中为 SWI-Prolog 提供语言服务器协议（LSP）支持。

在线帮助

7.1 library(help): 基于文本的手册

该模块提供 `help/1` 和 `apropos/1`。前者给出某个主题的帮助，后者在手册中搜索相关主题。

默认情况下，`help/1` 的结果会通过诸如 `less` 这样的分页器发送。该行为由以下内容控制：

- Prolog 标志 `help_pager`，可以设置为下列值之一：
- `false`

永不使用分页器。

- `default`

使用默认行为。系统会尝试判断 Prolog 是否运行在允许分页器的交互式环境中。如果是，它会检查环境变量 `PAGER`；否则尝试查找 `less` 程序。

- `Callable`

`Callable` 项会被解释为 `program_name(Arg, ...)`。例如，`less('-r')` 可以作为默认值。注意，如果使用单引号，程序名可以是绝对路径。

help

help(+What)

显示 `What` 的帮助。`What` 是一个描述帮助主题的项。`What` 的可用记法包括：

- `Atom`

最常用但也有歧义的形式，会显示所有匹配文档。例如：

```
?- help(append).
```

- `Name/Arity`

显示匹配 `Name/Arity` 的谓词帮助。`Arity` 可以未绑定。

- `Name//Arity`

显示匹配的 DCG 规则，也就是非终结符。

- `Module:Name`

显示 `Module` 中名为 `Name`、任意 `arity` 的谓词帮助。仅用于已载入代码。

- `Module:Name/Arity`

显示 `Module` 中名为 `Name` 且 `arity` 为 `Arity` 的谓词帮助。仅用于已载入代码。

- `f(Name/Arity)`

显示匹配的 Prolog 算术函数帮助。

- `c(Name)`

显示匹配的 C 接口函数帮助。

- `section(Label)`

显示手册中匹配 `Label` 的章节。

`help/1` 会显示手册中的文档，也会显示已载入用户代码中的文档，前提是该代码使用 `PIDoc` 编写了文档。若只想显示已载入谓词的文档，可以在谓词指示器前加上定义该谓词的模块。

如果精确匹配失败，该谓词会尝试模糊匹配；如果成功，会显示结果，并在开头给出警告，说明这些匹配基于模糊匹配。

如果可能，结果会通过诸如 `less` 的分页器发送。该行为由 Prolog 标志 `help_pager` 控制。参见本节开头的说明。

另请参阅 `apropos/1`，它用于搜索手册名称和摘要。

show_html_hook(+HTML:string)

这是一个半确定的 `multifile` 钩子，用于显示抽取出的 HTML 文档。如果该钩子失败，HTML 会使用 `html_text/2` 渲染为纯文本并显示在控制台上。

apropos(+Query)

打印手册中名称或摘要与 `Query` 匹配的对象。`Query` 可采用下列形式：

- `Type:Text`

查找与 `Text` 匹配的对象，并按 `Type` 过滤结果。`Type` 匹配采用大小写不敏感的前缀匹配。已定义的类型包括 `section`、`cfunction`、`function`、`iso_predicate`、`swi_builtin_predicate`、`library_predicate`、`dcg`，以及别名 `chapter`、`arithmetic`、`c_function`、`predicate`、`nonterminal` 和 `non_terminal`。例如：

```
?- apropos(c:close).
?- apropos(f:min).
```

- `Text`

`Text` 会被拆分为多个 `token`。如果某个主题的名称或摘要中包含所有 `token`，则认为匹配。匹配不区分大小写。结果会按匹配质量排序。

help_apropos(+Query, -Obj, -Summary, -Score)

在帮助数据库中查找匹配的已文档化对象。`Obj` 是正式对象标识符，`Summary` 是摘要说明，`Score` 是表示匹配质量的数字。

help_text(+Predicate:term, -HelpText:string)

如果 `Predicate` 是形如 `Name/Arity` 的项，且存在对应文档，则 `HelpText` 是从 HTML 帮助解析出的文本格式文档。

7.2 library(explain)：描述 Prolog 项

`library(explain)` 用于描述 Prolog 项。它最有用的功能是交叉引用能力。

```
?- explain(subset(_,_)).
"subset(_, _)" is a compound term
  from 2-th clause of lists:subset/2
  Referenced from 46-th clause of prolog_xref:imported/3
  Referenced from 68-th clause of prolog_xref:imported/3
lists:subset/2 is a predicate defined in
  /staff/jan/lib/pl-5.6.17/library/lists.pl:307
  Referenced from 2-th clause of lists:subset/2
  Possibly referenced from 2-th clause of lists:subset/2
```


注意，PceEmacs 可以跳转到定义，gxref/0 可用于查看依赖概览。

explain(@Term)

解释 Term。Term 可以是任意 Prolog 数据对象。某些项具有特殊含义：

- 对谓词的完整或部分引用，会给出谓词、它的主要属性以及对该谓词的引用。部分引用包括：
 - Module:Name/Arity
 - Module:Head
 - Name/Arity
 - Name//Arity
 - Name
 - Module:Name
- 某些谓词属性。这会列出具有该属性的谓词，谓词形式如上。该说明可以写成 Module:Property，也可以只写 Property。带模块限定的版本会把结果限制到 Module 中定义的谓词。支持的属性包括：
 - dynamic
 - thread_local
 - multifile
 - tabled

explain(@Term, -Explanation)

当 Explanation 是 Term 的解释时为真。Explanation 是一个元素列表，会使用 print_message(information, explain(Explanation)) 打印。

命令行历史

SWI-Prolog 提供一种查询替换机制，类似 Unix shell 中常见的历史替换。该功能是否可用由 `set_prolog_flag/2` 控制，使用 Prolog 标志 `history`。默认情况下，如果没有可用的交互式命令行编辑器，历史功能会启用。若要启用历史功能，并记住最近 50 条命令，可以把下面的内容放入启动文件：

```
:- set_prolog_flag(history, 50).
```

历史系统允许用户根据之前输入并被系统记住的查询组合出新的查询。可用的历史命令如下。若这些序列出现在带引号的原子或字符串中，则不会进行历史展开。

命令	说明
<code>!!.</code>	重复上一条查询
<code>!nr.</code>	重复编号为 <code><nr></code> 的查询
<code>!str.</code>	重复最近一条以 <code><str></code> 开头的查询
<code>h.</code>	显示命令历史
<code>!h.</code>	显示这份历史命令列表

复用顶层绑定

顶层目标成功执行后得到的绑定，如果它们不太大，会被断言到一个数据库中。这里的“不太大”由 Prolog 标志 `toplevel_var_size` 定义。这些值可以在后续顶层查询中以 `$Var` 的形式复用。如果后续查询使用了同名变量，系统会把该变量关联到最近一次绑定。示例：

```
1 ?- maplist(plus(1), `hello`, X).
X = [105,102,109,109,112].

2 ?- format('~s~n', [$X]).
ifmmp
true.

3 ?-
```

注意，也可以通过执行 `=/2` 来设置变量：

```
6 ?- X = statistics.
X = statistics.

7 ?- $X.
% Started at Fri Aug 24 16:42:53 2018
% 0.118 seconds cpu time for 456,902 inferences
% 7,574 atoms, 4,058 functors, 2,912 predicates, 56 modules, 109,791 VM-codes
%
%
%          Limit   Allocated   In use
% Local  stack:      -       20 Kb   1,888 b
% Global stack:      -       60 Kb    36 Kb
% Trail  stack:      -       30 Kb   4,112 b
%      Total:    1,024 Mb    110 Kb    42 Kb
%
% 3 garbage collections gained 178,400 bytes in 0.000 seconds.
% 2 clause garbage collections gained 134 clauses in 0.000 seconds.
% Stack shifts: 2 local, 2 global, 2 trail in 0.000 seconds
% 2 threads, 0 finished threads used 0.000 seconds
true.
```

调试器概览

像 C++、Python 或 JavaScript 这样的命令式语言，大多执行近似线性的代码，中间带有一些分支和子例程调用。它们的调试器支持逐行单步执行并在每一行暂停，或者运行程序直到命中断点后暂停。暂停时，用户可以检查当前程序状态，或向调试器发出命令。

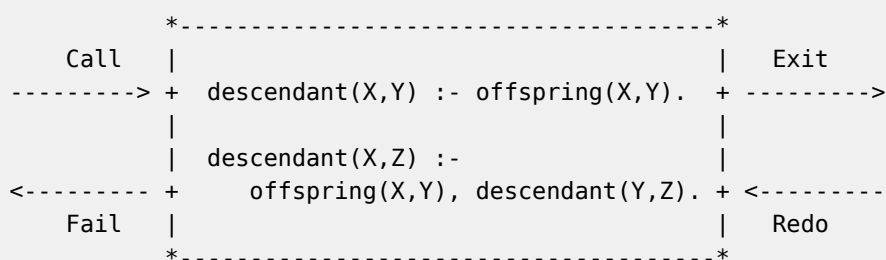
Prolog 的执行模型是逻辑式的：它会尝试证明逻辑谓词，因此需要不同的调试方式。SWI-Prolog 使用传统 Prolog 的“Byrd 盒模型”（Byrd Box Model）或“四端口模型”（4 Port Model）作为命令行调试器的基础，并在此基础上做了一些扩展。还有另外两个调试器也建立在这套基础设施上：一个是图形调试器，另一个是 [SWISH](#) 提供的 Web 界面中的远程调试。

所有可用于操纵调试器的谓词，其参考信息见调试器章节。

Byrd 盒模型和端口

标准 Prolog 调试工具围绕所谓的“Byrd 盒模型”或“四端口模型”构建。该模型把 Prolog 程序中的每个谓词建模为一个状态机，也就是一个“盒子”；程序求值时，这个盒子会在若干状态，也就是“端口”之间转换。开发者可以要求引擎在到达特定端口或特定谓词时暂停，以便检查程序。

阅读本概览时，请记住：“端口”只是“状态”的另一种说法；每个谓词在求值期间都会穿过这些状态。这个状态机被称为“盒子”，是因为它通常画成这样：



标准端口包括：call、redo、exit 和 fail。SWI-Prolog 在此基础上扩展了两个端口：unify 和 exception。每条 trace 都发生在谓词解析的某个特定阶段。回忆一下，在解析或“证明”一个谓词时，Prolog 引擎会：

1. 收集所有“可能”匹配的规则，也就是头部具有相同名称和参数数量的规则。- 如果有任何规则可能匹配，会 trace 一次 call。
- 当引擎回溯以寻找下一条匹配规则时，也会 trace redo。
1. 找到下一条其头部可以与该谓词合一的规则。- 如果找到这样的规则，会 trace unify，并显示合一结果。
- 如果没有任何规则头可以合一，则 trace fail。
1. 将合一得到的变量赋值应用到规则体中的子句，然后带着更新后的子句继续第 1 步。
2. 在匹配规则的所有规则体子句都成功、失败或抛出异常之后：- 如果它们全都成功，会 trace exit，表示该规则为真。
- 如果其中任何一个失败，会 trace fail，表示该规则为假。
- 如果其中任何一个抛出异常，会 trace exception。

这意味着，在某个谓词最初的 call 和该谓词 trace 结束之间，可能出现大量 trace。

Trace 模式示例

谓词 `trace/0` 会开启“trace 模式”。默认情况下，它会为每个谓词的每个端口产生 trace，并在每个端口暂停，以使用户检查程序状态。通常这是在 Prolog 控制台窗口中完成的；但对于嵌入式 Prolog 系统，或 Prolog 作为守护进程运行的情况，也可以通过 `libssh` 包获得提示符来完成。

注意：如果原生图形插件 XPCE 可用，命令 `gtrace/0` 和 `gspy/1` 会激活图形调试器，而 `tdebug/0` 和 `tspy/1` 允许调试任意线程。

每个目标都使用 Prolog 谓词 `write_term/2` 打印。打印风格由 Prolog 标志 `debugger_write_options` 定义；可以修改该标志，也可以使用 `tracer` 的 `w`、`p` 和 `d` 命令修改。

下面是一个展示基本流程的调试会话示例。默认情况下，`unify` 端口关闭，因为对命令行调试器来说，它在多数情况下不会提供很多额外信息。

```
is_a(rock1, rock).
is_a(rock2, rock).
color(rock1, red).

noun(X, Type) :- is_a(X, Type).
adjective(X, color, Value) :- color(X, Value).
```

```
?- trace.
true.

[trace] ?- noun(X, rock), adjective(X, color, red).
    Call: (11) noun(_9774, rock) ? creep
```

谓词 `trace/0` 打开了 trace 模式，现在每个提示符都会显示为 `[trace] ?-`。用户给出的初始查询是 `noun(X, rock), adjective(X, color, red)`，意思是寻找一个“红色的石头”。最后，触发的第一个端口是初始查询中第一个谓词的 `Call`，表示引擎将要寻找第一条匹配 `noun(_9774, rock)` 的规则。

按下空格键、`c` 或回车，会让 `tracer` 打印 `creep`，随后显示下一条 trace。`tracer` 还提供了许多其他命令，后文会介绍。

```
is_a(rock1, rock).
is_a(rock2, rock).
color(rock1, red).

noun(X, Type) :- is_a(X, Type).
adjective(X, color, Value) :- color(X, Value).
```

```
[trace] ?- noun(X, rock), adjective(X, color, red).
...
    Call: (12) is_a(_9774, rock) ? creep
    Exit: (12) is_a(rock1, rock) ? creep
```

```
Exit: (11) noun(rock1, rock) ? creep
...
```

接下来, noun/2 第一条子句的 is_a/2 调用得到 call trace, 因为引擎正在寻找下一条匹配 is_a(_9774, rock) 的规则。由于确实存在可以合一的事实 is_a(rock1, rock), trace 显示了 exit, 也就是成功, 并显示了该值。因为这是 noun/2 规则体中的最后一个谓词, 所以 noun/2 本身也得到一个 exit trace, 显示其头部的合一结果: noun(rock1, rock)。

```
is_a(rock1, rock).
is_a(rock2, rock).
color(rock1, red).

noun(X, Type) :- is_a(X, Type).
adjective(X, color, Value) :- color(X, Value).
```

```
[trace] ?- noun(X, rock), adjective(X, color, red).
...
Call: (11) adjective(rock1, color, red) ? creep
Call: (12) color(rock1, red) ? creep
Exit: (12) color(rock1, red) ? creep
Exit: (11) adjective(rock1, color, red) ? creep
X = rock1 ;
...
```

随后, Prolog 移动到初始查询中的下一个谓词 adjective/3, 并以类似方式求解它。由于这是查询中的最后一个谓词, 系统返回一个答案。按下 ; 会请求下一个答案, 并开始 Prolog 回溯。

```
is_a(rock1, rock).
is_a(rock2, rock).
color(rock1, red).

noun(X, Type) :- is_a(X, Type).
adjective(X, color, Value) :- color(X, Value).
```

```
[trace] ?- noun(X, rock), adjective(X, color, red).
...
Redo: (12) is_a(_9774, rock) ? creep
Exit: (12) is_a(rock2, rock) ? creep
Exit: (11) noun(rock2, rock) ? creep
Call: (11) adjective(rock2, color, red) ? creep
Call: (12) color(rock2, red) ? creep
Fail: (12) color(rock2, red) ? creep
Fail: (11) adjective(rock2, color, red) ? creep
false.
```

唯一需要 redo 的选择点, 也就是需要回溯的地方, 是 noun/2 中的 is_a/2 子句, 因为还有一个潜在匹配 is_a(rock2, rock) 可以尝试合一。它能够合一, 所以 trace 显示 exit; 同时也使 noun(rock2, rock) 像前面一样以 exit 成功。

随着 trace 继续，可以看到 color(rock2, red) 激活了 fail 端口，因为无法证明该谓词，所以整个查询最终返回 false。

在输入 notrace. 关闭 trace 模式之前，之后提出的每个查询都会继续被 trace。

Trace 模式选项：leash/1 和 visible/1

使用 trace/0 启用 trace 模式时，tracer 默认会在每个谓词命中的每个端口暂停，并等待命令。谓词 leash/1 可用于修改哪些端口会暂停。这是全局设置，因此修改后会一直保留，直到再次修改或 SWI-Prolog 重启。通过 notrace/0 禁用 tracer 不会影响哪些端口被 leash。

leash/1 的参数必须以 + 开头表示添加，或以 - 开头表示移除，后跟端口名称，例如 call、exit 等。也可以使用特殊项，例如 all，从而不必手动添加或移除每个端口。

如果只想在 fail 端口停止，可以这样使用 leash/1：

```
?- leash(-all).
true.

?- leash(+fail).
true.

?- trace.
true.

[trace] ?- noun(X, rock), adjective(X, color, red).
  Call: (11) noun(_3794, rock)
  Call: (12) is_a(_3794, rock)
  Exit: (12) is_a(rock1, rock)
  Exit: (11) noun(rock1, rock)
  Call: (11) adjective(rock1, color, red)
  Call: (12) color(rock1, red)
  Exit: (12) color(rock1, red)
  Exit: (11) adjective(rock1, color, red)
X = rock1 ;
  Redo: (12) is_a(_3794, rock)
  Exit: (12) is_a(rock2, rock)
  Exit: (11) noun(rock2, rock)
  Call: (11) adjective(rock2, color, red)
  Call: (12) color(rock2, red)
  Fail: (12) color(rock2, red) ? creep
  Fail: (11) adjective(rock2, color, red) ? creep
false.
```

现在，只有以 Fail: 开头的行后面带有 creep，因为那是 tracer 唯一暂停等待命令的时候。如果希望永不暂停，只想看到所有 trace，可以使用 leash(-all)，并且不要重新打开任何端口。

默认端口仍然会打印出来，因为另一个设置 visible/1 控制哪些端口会被打印。visible/1 的参数形式与 leash/1 相同。如果只想在 fail 端口停止并且只显示 fail 端口，可以这样使用 leash/1 和 visible/1：

```
?- leash(-all).
true.
```

```
?- leash(+fail).
true.

?- visible(-all).
true.

?- visible(+fail).
true.

?- trace.
true.

[trace] ?- noun(X, rock), adjective(X, color, red).
X = rock1 ;
    Fail: (12) color(rock2, red) ? creep
    Fail: (11) adjective(rock2, color, red) ? creep
false.
```

暂停时的 Trace 模式命令

当 tracer 在某个端口暂停时，除了按空格键之外，还可以做很多事情。所有动作都是单字符命令，并且无需等待回车就会执行，除非命令行选项 `--no-tty` 处于活动状态。暂停时按 `?` 或 `h` 也会打印这些命令的列表。

控制流命令

命令	键	说明
Abort	a	中止 Prolog 执行，参见 <code>abort/0</code>
Break	b	进入 Prolog break 环境，参见 <code>break/0</code>
Creep	c	继续执行，在下一个端口停止；也可以按回车或空格
Exit	e	终止 Prolog，参见 <code>halt/0</code>
Fail	f	强制当前目标失败
Find	/	搜索某个端口，见下文 Find 命令说明
Ignore	i	忽略当前目标，假装它已经成功
Leap	l	继续执行，在下一个 spy point 停止
No debug	n	以 no debug 模式继续执行
Repeat find	.	重复上一次 find 命令
Retry	r	撤销当前目标从 <code>call</code> 端口之后的所有动作，数据库和 I/O 动作除外，并从该目标的 <code>call</code> 端口恢复执行

Skip	s	继续执行，在当前目标的下一个端口停止，因此会跳过该目标子调用的所有端口
Spy	+	在当前谓词上设置 spy point，参见 spy/1
No spy	-	从当前谓词移除 spy point，参见 nospy/1
Up	u	继续执行，在父目标的下一个端口停止，因此会跳过当前目标及其所有子调用。该选项适合停止 trace 由失败驱动的循环

Find (/) 说明和示例

Find (/) 命令会继续执行，直到找到与 find 模式匹配的端口。输入 / 后，用户可以输入一行来指定要搜索的端口。该行由一组表示端口类型的字母组成，后面可以跟一个可选项；该项应与该端口正在运行的目标合一。如果没有指定项，则把它视为变量，也就是搜索指定类型的任意端口。如果给出原子，则任何函子名等于该原子的目标都会匹配。示例：

命令	说明
/f	搜索任意 fail 端口
/fe solve	搜索名称为 solve 的任意目标上的 fail 或 exit 端口
/c solve(a, _)	搜索对 solve/2 的调用，且第一个参数是变量或原子 a
/a member(_, _)	搜索 member/2 上的任意端口。这等价于在 member/2 上设置 spy point

信息命令

命令	键	说明
Alternatives	A	显示所有还有替代分支的目标
Goals	g	显示父目标列表，也就是执行栈。注意，由于尾递归优化，一些父目标可能已经不存在
Help	h	显示可用选项；也可以按 ?
Listing	L	使用 listing/1 列出当前谓词

格式化命令

命令	键	说明
Context	C	切换“显示上下文”。如果为 on，目标的上下文模块会显示在方括号中。默认值为 off

Display	d	设置 debugger_write_options 的 max_depth(Depth) 选项，限制打印项的深度。另请参阅 w 和 p 选项
Print	p	将 Prolog 标志 debugger_write_options 设置为 [quoted(true), portray(true), max_depth(10), priority(699)]。这是默认值
Write	w	将 Prolog 标志 debugger_write_options 设置为 [quoted(true), attributes(write), priority(699)]，绕过 portray/1 等

Trace 模式与 Trace Point

这里需要稍微绕开一下，说明几个容易混淆的相关谓词：如果只想 trace 一个谓词或一组选定谓词，可以使用 trace/1 或 trace/2 谓词设置 trace point。虽然它们使用同一个基础谓词名 trace，但这些谓词会忽略 leash/1 和 visible/1 的全局设置，并且在 trace 端口时不会暂停。它们实际上是另一个功能，只是也会输出 trace。

trace point 设置在某个特定谓词上，并 trace 该谓词的端口；无论当前是否处于 trace/0 的 trace 模式，它都会工作。如果使用 trace/2 变体，每个 trace point 可以 trace 不同端口。

```
?- trace(is_a/2).
%      is_a/2: [all]
true.

?- noun(X, rock), adjective(X, color, red).
T Call: is_a(_25702, rock)
T Exit: is_a(rock1, rock)
X = rock1 ;
T Redo: is_a(rock1, rock)
T Exit: is_a(rock2, rock)
false.
```

请注意：这里不需要使用 trace/0 打开 trace 模式；它只输出执行 is_a/2 时命中的端口；并且程序从未暂停。

实际上，如果在使用 trace point 时又打开 trace 模式，情况会变得非常混乱，因为 trace point 基础设施本身也会被 trace。

```
?- trace(is_a/2).
%      is_a/2: [all]
true.

?- trace.
true.

[trace] ?- noun(X, rock), adjective(X, color, red).
Call: (11) noun(_29318, rock) ? creep
Call: (12) is_a(_29318, rock) ? creep
Call: (13) print_message(debug, frame(user:is_a(_29318, rock), trace(call))) ?
creep
Call: (18) push_msg(frame(user:is_a(_29318, rock), trace(call))) ? creep
Call: (21) exception(undefined_global_variable, '$inprint_message', _30046) ?
```

```

creep
  Fail: (21) exception(undefined_global_variable, '$inprint_message', _30090) ?
creep
  Exit: (18) push_msg(frame(user:is_a(_29318, rock), trace(call))) ? creep
  Call: (19) prolog:message(frame(user:is_a(_29318, rock), trace(call)), _30140,
_30142) ? creep
  Fail: (19) prolog:message(frame(user:is_a(_29318, rock), trace(call)), _30140,
_30142) ? creep
  Call: (19) message_property(debug, stream(_30192)) ? creep
  Fail: (19) message_property(debug, stream(_30192)) ? creep
  Call: (20) message_property(debug, prefix(_30200)) ? creep
  Fail: (20) message_property(debug, prefix(_30200)) ? creep
T Call: is_a(_29318, rock)
  Call: (17) pop_msg ? creep
  Exit: (17) pop_msg ? creep
...Lots more after this...

```

所以，trace point 和 trace mode 是两个名称容易混淆、但彼此独立的功能。

Spy Point 和 Debug 模式

回到 trace 模式相关功能：由于 Prolog 程序的 trace 输出通常可能非常大，有时希望只在程序深处的某个特定点开始 trace 模式。这正是 spy point 的用途。它指定某个谓词，命中该谓词时应打开 trace 模式。

可以这样启用 spy point: `spy(mypredicate/2)`。执行该命令后，第一次遇到 `mypredicate/2` 时会打开 trace 模式，并像平常一样工作。这包括遵守全局的 `leash/1` 和 `visible/1` 设置。可以使用 `nospay/1` 或 `nospayall/0` 移除 spy point。

```

is_a(rock1, rock).
is_a(rock2, rock).
color(rock1, red).

noun(X, Type) :- is_a(X, Type).
adjective(X, color, Value) :- color(X, Value).

```

```

?- spy(is_a/2).
% Spy point on is_a/2
true.

[debug] ?- noun(X, rock), adjective(X, color, red).
* Call: (12) is_a(_1858, rock) ? creep
* Exit: (12) is_a(rock1, rock) ? creep
Exit: (11) noun(rock1, rock) ? creep
Call: (11) adjective(rock1, color, red) ? creep
Call: (12) color(rock1, red) ? creep
Exit: (12) color(rock1, red) ? creep
Exit: (11) adjective(rock1, color, red) ? creep
X = rock1 ;
* Redo: (12) is_a(_1858, rock) ? creep
* Exit: (12) is_a(rock2, rock) ? creep
Exit: (11) noun(rock2, rock) ? creep
Call: (11) adjective(rock2, color, red) ? creep

```

```

Call: (12) color(rock2, red) ? creep
Fail: (12) color(rock2, red) ? creep
Fail: (11) adjective(rock2, color, red) ? creep
false.

```

命中 spy point 后，上面的输出与对初始查询运行 `trace/0` 得到的 trace 相同，只是显然缺少 spy point 之前的所有 trace。

注意，调用 `spy/1` 后，`?-` 前面会出现一个新标签，也就是 `[debug]`：

```

?- spy(is_a/2).
% Spy point on is_a/2
true.

[debug] ?-

```

这表示系统处于“debug 模式”。debug 模式做两件事：告诉系统监视 spy point；关闭一些会让 trace 难以理解的优化。许多 Prolog 书籍中描述的理想四端口模型，在许多 Prolog 实现中不可见，因为代码优化会移除一部分选择点和 exit 点。如果某个目标确定性成功，或其替代分支被 cut 移除，回溯点不会显示。运行在 debug 模式时，选择点只有在被 cut 移除时才会销毁，并且最后调用优化会关闭。注意：这意味着系统在 debug 模式下可能耗尽栈，而非 debug 模式下不会出现问题。

可以使用 `nodebug/0` 再次关闭 debug 模式，但这样 spy point 会被忽略，虽然仍会被记住。通过 `debug/0` 重新打开 debug 模式后，会再次命中 spy point。

```

is_a(rock1, rock).
is_a(rock2, rock).
color(rock1, red).

noun(X, Type) :- is_a(X, Type).
adjective(X, color, Value) :- color(X, Value).

```

```

?- spy(is_a/2).
% Spy point on is_a/2
true.

[debug] ?- nodebug.
true.

?- noun(X, rock).
X = rock1 ;
X = rock2.

?- debug.
true.

[debug] ?- noun(X, rock).
* Call: (11) is_a(_47826, rock) ? creep
* Exit: (11) is_a(rock1, rock) ? creep
Exit: (10) noun(rock1, rock) ? creep

```

```

X = rock1 ;
* Redo: (11) is_a(_47826, rock) ? creep
* Exit: (11) is_a(rock2, rock) ? creep
  Exit: (10) noun(rock2, rock) ? creep
X = rock2.

```

因此，debug 模式允许 Prolog 监视 spy point，并在命中 spy point 时启用 trace 模式。谓词 `tracing/0` 和 `debugging/0` 会报告系统是否处于这些模式之一。

断点

有时候，连 spy point 也不够。某个谓词可能在许多地方使用，而只在某一次特定调用发生时打开 trace 模式会更有帮助。断点允许在命中特定源文件、行号以及该行中的字符位置时打开 trace 模式。使用的谓词是 `set_breakpoint/4` 和 `set_breakpoint/5`。可以同时激活多个断点。

注意，这些谓词提供的接口并不面向最终用户。内置的 PceEmacs 编辑器也嵌入在图形调试器中，它允许根据光标位置设置断点。

现在，`Example.pl` 修改为包含多个对 `noun/2` 的调用：

```

is_a(rock1, rock).
is_a(rock2, rock).
color(rock1, red).

noun(X, Type) :- is_a(X, Type).
adjective(X, color, Value) :- color(X, Value).
test_noun1(X, Type) :- noun(X, Type).
test_noun2(X, Type) :- noun(X, Type).

```

如果只想在 `noun/2` 从 `test_noun2/2` 调用时启用 trace，可以这样使用 `set_breakpoint/4`：

```

?- set_breakpoint('/...path.../Example.pl', 8, 24, ID).
% Breakpoint 1 in 1-st clause of test_noun2/2 at Example.pl:8
ID = 1.

?- debug.
true.

[debug] ?- noun(X, rock).
X = rock1 .

[debug] ?- test_noun1(X, rock).
X = rock1 .

[debug] ?- test_noun2(X, rock).
  Call: (11) noun(_44982, rock) ? creep
  Call: (12) is_a(_44982, rock) ? creep
  Exit: (12) is_a(rock1, rock) ? creep
  Exit: (11) noun(rock1, rock) ? creep
  Exit: (10) test_noun2(rock1, rock) ? creep
X = rock1 .

[trace] ?- notrace.

```

```
true.

[debug] ?-
```

调用 `set_breakpoint/4` 时必须指定源文件 `Example.pl`、行号 8 和该行中的字符位置 24，以精确指出哪条子句应该打开 `trace` 模式。使用图形调试器时这会容易得多，因为它会显示源代码。

如果系统不在 `debug` 模式下，断点不会触发。与设置 `spy point` 不同，`set_breakpoint/4` 不会自动进入 `debug` 模式。因此，示例中使用 `debug/0` 手动打开了 `debug` 模式。

输出显示，只有对 `test_noun2/2` 的调用，也就是设置断点的位置，实际打开了 `trace` 模式。注意，结尾处的 `[trace] ?-` 表明 `trace` 模式在触发后仍保持开启。可以通过 `notrace/0` 再次关闭它，这会让系统留在 `debug` 模式。调用 `nodebug/0` 可以一次关闭所有调试模式，因为关闭 `debug` 模式会自动关闭 `trace` 模式。

此外，SWI-Prolog 支持通过 `set_breakpoint_condition/2` 为每个断点附加任意目标，从而得到条件断点。条件断点与前面讨论的普通断点相同，只是每当断点被触发时，系统会调用给定目标；只有在该目标成功时，`trace` 模式才会打开。

如果只想在 `noun/2` 从 `test_noun2/2` 调用，并且第一个参数为 `rock2` 时启用 `trace`，可以如下使用 `set_breakpoint_condition/2`。注意，条件是一个 Prolog 字符串；系统会解析该字符串以获得目标以及变量名。得到的目标会在执行该子句体的模块中调用，参见 `clause_property/2` 的 `module` 属性。

```
?- set_breakpoint('/...path.../Example.pl', 8, 24, ID).
ID = 1.

?- set_breakpoint_condition(1, "X == rock2").
true.

?- debug.
true.

[debug] ?- test_noun2(X, rock).
X = rock1 ;
X = rock2.

[debug] ?- test_noun2(rock2, rock).
  Call: (11) noun(rock2, rock) ? creep
  Call: (12) is_a(rock2, rock) ? creep
  Exit: (12) is_a(rock2, rock) ? creep
  Exit: (11) noun(rock2, rock) ? creep
  Exit: (10) test_noun2(rock2, rock) ? creep
true.

[trace] ?-
```

命令行调试器总结

总结来说，实际上存在两个不同的“tracing”功能：`trace` 模式和 `trace point`。两者都会使用 Byrd 盒模型向控制台写出 `trace`，但相似之处到此为止。

Trace 模式

Trace 模式是主要的 Prolog 命令行调试器。它可以 trace 谓词穿过解析状态的过程，这些状态在 Byrd 盒模型中表示为端口；也可以在命中特定端口时暂停，让用户发出命令。

可以通过 `trace/0` 手动打开 trace 模式。也可以在使用 `debug/0` 进入 debug 模式后，通过 `spy/1` 在遇到某个特定谓词时打开；或者通过 `set_breakpoint/4`、`set_breakpoint/5` 在遇到某个谓词的特定调用时打开。

处于 trace 模式时，`visible/1` 控制哪些端口会写到控制台，`leash/1` 控制哪些端口会导致执行暂停，以便检查程序。

执行暂停时，可以使用许多命令检查程序状态、让目标失败或成功，等等。

Trace 模式通过 `notrace/0` 关闭，debug 模式通过 `nodebug/0` 关闭。

Trace Point

Trace point 是一个独立于 trace 模式的功能，它允许在某个谓词求值时把指定端口写到控制台。它永远不会暂停程序执行，也不需要处于 trace 或 debug 模式。

Trace point 通过 `trace/1` 和 `trace/2` 打开。

它们不关注 `visible/1`，因为显示哪些端口是在 `trace/2` 中设置的；也不关注 `leash/1`，因为它们不会暂停执行。

可以通过 `trace/2` 关闭它们。

加载和运行项目

大多数 Prolog 程序会拆分到多个文件中，这些文件组织在一个目录里，也可以再分到多个子目录中。通常，所有文件都是 Prolog 模块（module）文件，参见模块章节。这个目录通常包含一个文件，常命名为 `load.pl`，它会使用 `use_module/[1,2]` 载入所有其他文件（模块）；对于不使用模块的项目，则使用 `ensure_loaded/1`。

如果项目是应用程序，而不是库，则有几种启动方式。一种选择是使用命令行选项 `-g goal`。经典 Prolog 方式是使用 `initialization/1` 指令。后一种方式的问题在于，这类指令既用于模块中的运行时初始化，也用于启动应用程序，而它们的执行顺序很难控制。因此，SWI-Prolog 引入了 `initialization/2`，增加一个参数来指定初始化的角色，并间接指定初始化顺序。现在，应用程序入口点可以这样声明：

```
:- initialization(start, main).

start :-
    ...
```

采用这些约定后，可以用下面的命令行运行应用程序。其中 `option ...` 是 Prolog 选项，用来控制例如内存限制等设置；通常不需要指定。`arg ...` 会通过 Prolog 标志 `argv` 提供给程序。

```
% swipl [option ...] load.pl [arg ...]
```

如果只想载入代码而不运行应用程序，并且入口点是通过上面所述的 `initialization/2` 指令启动的，则可以使用 `-l`。载入之后，可以调试和/或编辑应用程序。

```
% swipl [option ...] -l load.pl [arg ...]
```

应用程序通常不会像上面那样直接使用 `start/0`，而是使用库 `library(main)` 中的 `main/0`。谓词 `main/0` 会为非开发场景做好准备，并用应用程序的 `argv`（命令行参数）调用 `main/1`。这些参数通常会使用同一库中的 `argv_options/2` 处理为位置参数（positional arguments）和选项（options）。

上面的方式在命令行中使用 Prolog 时可以很好工作，但在难以控制 SWI-Prolog 命令行的场景中不太适合，例如使用 `swipl-win`，或在 Emacs 等 IDE 下运行 Prolog。把一个使用上述 `initialization/2` 指令的程序载入顶层：

```
?- [load].
```

不会启动入口点。而使用 `swipl-win` 打开 `.pl` 文件则会启动入口点。

运行应用程序

如果想让程序可用于真实场景，有多种选择。最佳选择取决于程序是否只在装有 SWI-Prolog 开发系统的机器上使用、程序大小，以及操作系统（Unix 或 Windows）。有四种选择：

- 在类 Unix 系统上，可以使用 shebang 魔法序列把 Prolog 源文件变成可执行文件。参见“使用 PrologScript”。
- 在任何系统上，都可以使用 shell 脚本（Unix 的 `sh` 或 Windows 的 `cmd`）来启动应用程序。参见“创建 shell 脚本”。

- 在任何系统上，都可以创建一个保存状态 (saved state)，其中包含虚拟机代码和启动序列。保存状态可以是独立的；在采取一些预防措施后，它们也可以在未安装 SWI-Prolog 本身的环境中工作。它们启动很快，但体积较大；如果程序使用原生代码扩展和文件资源，从该程序创建保存状态并不简单，具体细节取决于操作系统和所需资源。参见“创建保存状态”。
- 在任何系统上，都可以把一个 Prolog 文件加入指定目录，并允许用下面的形式启动它：

```
swipl name [arg ...]
```

可以通过 Prolog 包 (pack)、用户专属目录，或系统级目录，向 Prolog 安装添加新命令。参见“SWI-Prolog 应用脚本”。

使用 PrologScript

Prolog 源文件可以用 Unix 的 `#!` 魔法开头直接作为 Unix 程序使用。Unix 的 `#!` 魔法是允许的，因为如果 Prolog 文件的第一个字符是 `#`，第一行会被视为注释。`#` 号也可以合法地作为普通 Prolog 子句的开头。在极少数确实需要这种写法的情况下，请把第一行留空，或添加一个头部注释。要创建 Prolog 脚本，可以使用下面两种形式之一作为第一行。第一种可把脚本绑定到某个特定的 Prolog 安装，后一种则使用 `$PATH` 中默认安装的 Prolog。

```
#!/path/to/swipl
#!/usr/bin/env swipl
```

不同 Unix 派生系统对 HashBang 行中可执行程序参数的解释不同。为了可移植性，`#!` 后面必须立即跟一个可执行文件的绝对路径，并且应当没有参数或只有一个参数。可执行文件路径和参数都不能使用引号或空格。以这种方式启动时，Prolog 标志 `argv` 包含脚本调用之后的命令行参数。

从 7.5.8 版本开始，`initialization/2` 支持 `When` 选项 `program` 和 `main`，因此可以如下定义一个 Prolog 脚本，用于求值命令行中的算术表达式。注意，`main/0` 由库 `library(main)` 定义。它会在禁用信号处理之后，用命令行参数调用 `main/1`。

```
#!/usr/bin/env swipl

:- initialization(main, main).

main(Argv) :-
    atomic_list_concat(Argv, ' ', SingleArg),
    term_to_atom(Term, SingleArg),
    Val is Term,
    format('~w~n', [Val]).
```

下面是两个运行示例：

```
% ./eval 1+2
3
% ./eval foo
ERROR: is/2: Arithmetic: `foo/0' is not a function
```

出于调试或检查目的，可以使用 `-l` 或 `-t` 启动 Prolog 脚本。例如，`-l` 只会载入脚本，忽略 `main` 和 `program` 初始化。

```
swipl -l eval 1+1
<banner>

?- main.
2
true.

?-
```

也可以使用 `-t prolog` 强制程序在应用程序完成后进入交互式顶层：

```
swipl -t prolog eval 1+1
2
?-
```

Windows 版本会直接忽略 `#!` 行。旧版本曾经从 HashBang 行中提取命令行参数。从 5.9 版本开始，所有相关设置都可以通过指令完成。由于 HashBang 行处理存在兼容性问题，SWI-Prolog 决定完全移除这项处理。

创建 shell 脚本

随着 PrologScript 的引入（参见“使用 PrologScript”），本节说明的 shell 脚本方式对于大多数应用程序已经显得多余。

尤其是在 Unix 系统上，对于规模不太大的应用程序，编写一个只负责载入应用程序并调用入口点的 shell 脚本，通常是不错的选择。下面先给出脚本骨架，随后给出获取程序参数的 Prolog 代码。详情参见库 `library(main)` 和 `argv_options/3`。

```
#!/bin/sh

base=<absolute-path-to-source>
SWIPL=swipl

exec $SWIPL "$base/load.pl" -- "$@"
```

```
:- use_module(library(main)).
:- initialization(main,main).

main(Argv) :-
    argv_options(Argv, Positional, Options),
    go(Positional, Options).

go(Positional, Options) :-
    ...
```

在 Windows 系统上，可以通过创建指向 Prolog 的快捷方式并传入适当选项，或编写 `.bat` 文件，达成类似行为。

创建保存状态

对于较大的程序，以及必须在没有安装 SWI-Prolog 开发系统的机器上运行的程序，创建保存状态是最佳方案。保存状态使用 `qsave_program/[1,2]` 或命令行选项 `-c` 创建。保存状态是一个

文件，包含机器无关的中间代码，并采用专为快速载入设计的格式。保存状态不依赖 CPU 指令集或字节序。32 位和 64 位的保存状态不兼容。通常，保存状态只能在创建它的同一 Prolog 版本上运行。也可以把仿真器集成到保存状态中，从而创建一个单文件但机器相关的可执行文件。这个过程会在运行时章节中说明。

使用 -c 命令行选项进行编译

该机制会载入一系列 Prolog 源文件，然后像 `qsave_program/2` 一样创建保存状态。命令语法如下：

```
% swipl [option ...] [-o output] -c file.pl ...
```

参数 `options` 是传给 `qsave_program/2` 的选项，写成下面的格式。选项名称及其值由 `qsave_program/2` 说明。

```
--option-name=option-value
```

例如，要创建一个独立可执行文件，使其启动时执行 `main/0`，并且通过 `load.pl` 载入源代码，可以使用下面的命令：

```
% swipl --goal=main --stand_alone=true -o myprog -c load.pl
```

这与执行下面的内容效果完全相同：

```
% swipl
<banner>

?- [load].
?- qsave_program(myprog,
    [ goal(main),
      stand_alone(true)
    ]).
?- halt.
```

SWI-Prolog 应用脚本

从 9.1.18 版本开始，SWI-Prolog 允许用下面的命令启动应用程序：

```
swipl [option ...] [path:]name [arg ...]
```

这条命令行首先处理“命令行选项”中说明的 Prolog 选项。注意，大多数标准 Prolog 命令行选项在这里并不相关。`-f` 默认值为 `none`，这意味着默认不会载入用户初始化文件。如果应用程序希望载入用户初始化文件，应在该文件存在时载入 `user_app_config(init)`（参见 `exists_source/1`）。

接下来，它会使用 SWI-Prolog 的文件搜索机制（由 `absolute_file_name/3` 定义）定位 `path(name)`。载入这个文件后，它会查找通过 `initialization/2` 为 `main` 注册的最后一个目标，如本章前文所述；如果没有 `main` 的初始化指令，程序会以错误终止。默认情况下，入口点终止后应用程序也会终止。入口点可以通过调用 `cli_enable_development_system/0` 启用交互式 Prolog REPL 循环。除了 `main` 之外，也允许使用其他形式的 `initialization/2` 指令。

SWI-Prolog 中文文档

[path:]name 后面的所有命令行选项，都可以通过 Prolog 标志 argv 访问。

可选的 path 默认为 app。默认情况下，应用会在下面这些目录中搜索。详情参见 file_search_path/2。

1. SWI-Prolog 安装目录中的 app 目录。
2. 用户和站点配置。在使用 XDG 文件名约定的 POSIX 系统上，这通常是 ~/.local/share/swi-prolog/app/ 和 /usr/share/swi-prolog/app。
3. Prolog 包 (pack) 的 app 目录。

安装会提供下列应用：

app

打印已安装应用的信息。例如，要列出所有可用应用，运行：

```
swipl app list
```

pack

Prolog 包的命令行管理工具。这是 Prolog 库 library(prolog_pack) 的前端。例如，要查找与 *type* 相关的包，可以使用下面的命令：

```
swipl pack find type
```

环境控制（Prolog 标志）

谓词 `current_prolog_flag/2` 和 `set_prolog_flag/2` 允许用户检查和修改执行环境。它们提供对安装特性、可选功能、操作系统、外部代码环境、命令行参数、版本信息的访问，也提供一些运行时标志，用于控制特定谓词的运行时行为，从而兼容其他 Prolog 环境。

`current_prolog_flag/2`

`current_prolog_flag(?Key, -Value)` 定义了访问安装特性的接口，包括编译进系统的选项、版本、home 目录等。如果两个参数都未绑定，它会生成所有已定义的 Prolog 标志。如果 Key 已实例化，它会把 Value 与该 Prolog 标志的值合一；如果 Key 不是 Prolog 标志，则失败。

标记为可变（changeable）的标志可以由用户使用 `set_prolog_flag/2` 修改。标志值有类型。标记为 `bool` 的标志可以取值 `true` 或 `false`。谓词 `create_prolog_flag/3` 可用于创建描述或控制库和应用程序行为的标志。库 `library(settings)` 为管理应用程序参数提供了另一种接口。

有些 Prolog 标志并不是在所有版本中都定义。下文通常会用“如果存在且为 `true`”来说明这种情况。一个布尔 Prolog 标志为 `true`，当且仅当该 Prolog 标志存在，并且 Value 是原子 `true`。对这类标志的测试应写成下面这样：

```
(  current_prolog_flag(windows, true)
->  <Do MS-Windows things>
;   <Do normal things>
)
```

有些 Prolog 标志的作用域限定在一个源文件内。这意味着，如果在文件内部用指令设置它们，载入该文件开始时遇到的标志值会在载入完成后恢复。目前，下列标志具有源文件作用域：`generate_debug_info` 和 `optimise`。

新线程会复制创建它的线程（即父线程）的所有标志。这使用 `copy-on-write` 技术实现。因此，在线程内部修改某个标志不会影响其他线程。

`abi_version(dict, r)`

该标志的值是一个 `dict`，其中的键描述各种应用程序二进制接口（Application Binary Interface, ABI）组件的版本。详情见“二进制兼容性”。

`access_level(atom, rw)`

该标志定义普通“用户”视图（`user`，默认值）或“系统”视图。在系统视图中，所有系统代码都可以像普通用户代码一样完全访问。在用户视图中，某些操作不被允许，某些细节会保持不可见。具体后果没有完全定义；例如，在系统访问级别下可以跟踪系统代码，也可以重定义系统谓词。

`address_bits(integer, r)`

宿主机器的地址大小，通常为 32 或 64。除了最大堆栈限制之外，这对用户几乎没有影响。另请参阅 Prolog 标志 `arch`。

`agc_close_streams(boolean, rw)`

当为 `true` 时，原子垃圾回收器会关闭那些仍处于打开状态但已被垃圾回收的流，并打印警告。默认值为 `false`，未来版本可能改为 `true`。下面是这类警告示例：

```
WARNING: AGC: closed <stream>(0x560e29014400)
```

注意，不应把关闭 I/O 流的工作留给原子垃圾回收器。原因是原子垃圾回收器可能要过很久才运行，而且它是保守的（conservative），并不保证所有垃圾原子都能被回收。使用 I/O 流的代码应使用 `setup_call_cleanup/3`，结构如下，其中 `process/1` 是从 Stream 读取或写入的谓词。

```
setup_call_cleanup(
    open(..., Stream),
    process(Stream),
    close(Stream)),
...
```

注意，该标志在 `main` 线程中的设置会生效。

agc_margin (integer, rw)

如果可能成为垃圾的原子数量达到这个值，则在第一个机会执行原子垃圾回收。初始值为 10,000，可以修改。值为 0 会禁用原子垃圾回收。另请参阅 `PL_register_atom()`。由于 SWI-Prolog 对原子长度没有限制，10,000 个原子仍可能占用大量内存。使用超大原子的应用程序可能希望显式调用 `garbage_collect_atoms/0`，或降低这个阈值。

allow_dot_in_atom (bool, rw)

如果为 `true`（默认值为 `false`），未加引号且以字母开头的原子中可以嵌入点号。嵌入的点号后面必须跟一个标识符继续字符，也就是字母、数字或下划线。许多语言允许在标识符中使用点号，因此这个标志对定义 DSL 可能有用。注意，这会与级联函数式记法冲突。例如，如果该标志设置为 `true`，`Post.meta.author` 会被读取为 `.(Post, 'meta.author')`。

allow_variable_name_as_functor (bool, rw)

如果为 `true`（默认值为 `false`），`Functor(arg)` 会被读取为 `'Functor'(arg)`。有些应用程序使用 Prolog 的 `read/1` 谓词读取应用程序自定义的脚本语言。在这些场景中，常常很难向非 Prolog 用户解释常量和函数只能以小写字母开头。可以通过带 `variable_names` 选项调用 `read_term/2`，并把变量绑定到它们的名称，把变量转换为以大写字母开头的原子。借助此功能，`F(x)` 可以变成这类脚本语言中的合法语法。该特性由 Robert van Engelen 建议，属于 SWI-Prolog 特有功能。

android (bool, r)

如果存在且为 `true`，表示正在 Android 操作系统上运行。其他操作系统上不存在该标志。

android_api (integer, r)

如果在 Android 上运行，该标志表示 C 宏 `__ANDROID_API__` 定义的编译期 API Level。在其他操作系统上未定义。这个 API level 不一定与运行设备的 API level 匹配，因为它表示编译时的 API level。

answer_write_options (term, rw)

交互式顶层使用该标志打印绑定（答案）的值。打印查询答案时，标志值会传给 `write_term/2`。默认值为 `[quoted(true), portray(true), max_depth(10), attributes(portray)]`。

atom_normalize_hook (bool, r)

当内核原子规范化 hook 处于活动状态时为 true。该 hook 由 PL_atom_normalize_hook() 注册，并由 library(unicode) 使用。读取器会用它在精确 NFC 检查和基于 wcwidth 的后备 NFC 检查之间做选择。另请参阅 unicode_atoms。

apple (bool, r)

如果存在且为 true，操作系统是 MacOSX。若用于编译该 SWI-Prolog 版本的 C 编译器定义了 __APPLE__，则会定义该标志。注意，在 MacOSX 上也会定义 unix 标志。

apple_universal_binary (bool, r)

如果存在且为 true，SWI-Prolog 构建为通用二进制 (universal binary)。通用二进制包含多个体系结构的原生可执行代码。目前支持的体系结构是 x86_64 和 arm64。组件的体系结构前缀是 fat-darwin，而 arch 取决于实际 CPU 类型。

arch (atom, r)

SWI-Prolog 正在运行的硬件和操作系统标识符。它用于为正确体系结构选择外部文件。另请参阅共享库相关章节和 file_search_path/2。在 Apple 平台上，另请参阅 apple_universal_binary。

argv (list, rw)

一个原子列表，表示应用程序命令行参数。应用程序命令行参数是 Prolog 初始化期间未处理的那些参数。注意，Prolog 的参数处理会在 -- 或第一个非选项参数处停止。另请参阅 os_argv。在 6.5.2 之前，argv 的定义等同于现在的 os_argv。出于兼容性和实用性的原因，后来修改为当前定义。

associated_file (atom, r)

如果 Prolog 启动时以某个 Prolog 文件作为参数，则设置该标志。例如，edit/0 会使用它编辑初始文件。

autoload (atom, rw)

该标志控制基于 autoload/1、autoload/2 以及自动加载库 (autoload libraries) 的谓词自动加载。它有下列取值：

- false: 谓词永不自动加载。如果谓词之前通过 autoload/[1,2] 导入，则立即用 use_module/[1,2] 载入引用的文件。注意，很多开发工具（例如 listing/1）必须先显式导入，才能在顶层使用。
- explicit: 不从自动加载库中自动加载，但对通过 autoload/[1,2] 导入的谓词使用惰性加载。
- user: 类似 false，但会把库谓词自动加载到全局 user 模块。这会让开发工具和库隐式可用于顶层，但不适用于模块。
- user_or_explicit: 组合 explicit 和 user，既为通过 autoload/[1,2] 导入的谓词提供惰性加载，又让整个库对顶层隐式可访问。
- true: 在所有地方提供完整自动加载。这是默认值。

back_quotes (codes, chars, string, symbol_char, rw)

定义反引号内容的项表示形式。默认值为 codes。如果给出 --traditional，默认值为 symbol_char，这允许在由符号组成的运算符中使用反引号。旧版本有一个布尔标志 backquoted_strings，用于在 string 和 symbol_char 之间切换。另请参阅字符串章节。

backtrace (bool, rw)

如果为 true（默认值），在未捕获异常上打印回溯。

backtrace_depth (integer, rw)

如果启用了错误回溯，该标志定义打印的最大栈帧数。默认值为 20。

backtrace_goal_depth (integer, rw)

回溯栈帧会在对目标做浅拷贝后打印。该标志决定目标项被拷贝的深度。默认值为 3。

backtrace_show_lines (bool, rw)

如果为 true（默认值），尝试重建异常发生处的行号。

bounded (bool, r)

ISO Prolog 标志。如果为 true，整数表示受 min_integer 和 max_integer 约束。如果为 false，整数可以任意大，且 min_integer 和 max_integer 不存在。标志 max_integer_size 可用于强制任意限制，而不是耗尽内存。参见算术类型相关章节。

break_level (integer, r)

当前 break level。通过 -t 启动的初始顶层值为 0。参见 break/0。未运行顶层循环的线程没有该标志。

build_type (atom, r)

该标志表示构建此 SWI-Prolog 实例时的 CMake CMAKE_BUILD_TYPE。可能值取决于平台。一些常见值包括 Debug、Release、MinSizeRel、RelWithDebInfo、Sanitize、DEB 或 PG0。

bundle (bool, r)

当 SWI-Prolog 作为独立 bundle 安装时为 true。SWI-Prolog 下载页分发的 Windows 和 MacOS 二进制包都会设置该标志。它用于调整文件搜索配置。

c_cc (atom, rw)

用于编译 SWI-Prolog 的 C 编译器名称，通常是 gcc、clang 或 cc。参见 swipl-ld 相关章节。

c_cflags (atom, rw)

用于编译 SWI-Prolog 的 CFLAGS。参见 swipl-ld 相关章节。

c_cxx (atom, rw)

用于测试 SWI-Prolog C++ 绑定的 C++ 编译器名称。这也是 swipl-ld 使用的默认 C++ 编译器，并用于通过默认设置编译 pack。注意，SWI-Prolog 本身不包含 C++ 代码，C++ 绑定只由头文件组成。因此不会出现 C++ ABI 兼容性问题。

c_ldflags (atom, rw)

用于链接 SWI-Prolog 的 LDFLAGS。参见 swipl-ld 相关章节。

c_libplso (atom, rw)

把扩展（共享对象或 DLL）链接到 SWI-Prolog 所需的库。在 ELF 系统上通常为空，在基于 COFF 的系统上通常为 -lswipl。参见 swipl-ld 相关章节。

c_libs (atom, rw)

链接嵌入 SWI-Prolog 的可执行文件所需的库。如果 SWI-Prolog 内核是共享库（DLL），通常为 -lswipl。如果 SWI-Prolog 内核位于静态库中，该标志还包含其依赖项。

char_conversion (bool, rw)

决定读取项时是否执行字符转换。另请参阅 `char_conversion/2`。

character_escapes (bool, rw)

如果为 `true`（默认值），`read/1` 会在带引号的原子和字符串中解释 \ 转义序列。可以修改。该标志局部于修改它的模块。参见字符转义语法章节。

character_escapes_unicode (bool, rw)

如果为 `true`（默认值），`write/1` 及相关谓词会使用 \uXXXX 或 \UXXXXXXXX 语法写出转义字符，而不是 ISO Prolog 的 \x<hex>\ 语法。SWI-Prolog 可以读取两种形式。

ci_speedup (float, rw)

如果某个哈希用于子句索引时至少达到该标志指定的加速比，则考虑生成该哈希。默认值为 1.5。

ci_max_var_fraction (float, rw)

如果某个参数在超过该比例的子句中未绑定，则不为该参数创建子句索引哈希表。默认值为 0.1。

ci_min_speedup_ratio (float, rw)

如果多参数哈希至少达到该值指定的效率，则考虑添加它。默认值为 3.0。

ci_max_lookahead (integer, rw)

如果找到一个子句，则在子句列表中最多向前扫描该数量的子句，以寻找可能的替代匹配。默认值为 100。

ci_min_clauses (integer, rw)

如果主索引参数（第一个参数）已实例化，当谓词拥有超过该数量的子句时，仍考虑使用哈希。默认值为 10。

cmake_build_type (atom, ro)

提供构建此 SWI-Prolog 版本时使用的 CMake build type。

colon_sets_calling_context (bool, ro)

使用结构 `Module:Goal` 会为执行 `Goal` 设置调用上下文。该标志由 ISO/IEC 13211-2（Prolog 模块标准）定义。参见模块章节。

color_term (bool, rw)

该标志由库 `library(ansi_term)` 管理。如果下面两个条件都为真，该库会在启动时载入。注意，这意味着从系统或个人初始化文件中把该标志设置为 `false` 会禁用彩色输出。谓词 `message_property/2` 可用于传给 `print_message/2` 的消息类型控制实际配色方案。

- `stream_property(current_output, tty(true))`
- `\+ current_prolog_flag(color_term, false)`

compile_meta_arguments (atom, rw)

该标志控制传给标记为 `θ` 或 `^` 的元调用的参数如何编译（参见 `meta_predicate/1`）。支持下列取值：

- `false`：默认值。元参数原样传递。如果参数是控制结构（例如 `(A,B)`、`(A;B)`、`(A->B;C)` 等），则在调用元谓词时将其编译为分配在环境栈上的临时子句。

- **control**: 把包含控制结构的元参数编译为辅助谓词。这通常能改善性能和调试体验。
- **always**: 总是创建中间子句, 即使对系统谓词也是如此。将来这可用于把生成谓词的普通头替换为特殊引用, 类似 `assert/2` 使用的数据库引用, 从而直接访问可执行代码, 避免元调用的运行时谓词查找。

compiled_at (atom, r)

描述系统的编译时间。只有用于编译 SWI-Prolog 的 C 编译器提供 `__DATE__` 和 `__TIME__` 宏时才可用。

conda (bool, r)

在 Conda 环境中构建时设置为 `true`。

console_menu (bool, r)

当 I/O 绑定到 Epilog (`swipl-win`) Prolog 控制台时设置为 `true`, 表示该控制台支持菜单。另请参阅 Epilog API 章节。

cpu_count (integer, rw)

系统中的物理 CPU 或核心数量。该标志标记为读写, 是为了允许假装系统拥有更多或更少处理器。另请参阅 `thread_setconcurrency/2` 和库 `library(thread)`。如果系统上无法获取 CPU 数量, 则该标志不可用。该标志不会包含在保存状态中 (参见 `qsave_program/1`)。

dde (bool, r)

如果该 Prolog 实例支持 DDE, 则设置为 `true`。

debug (bool, rw)

打开或关闭调试模式。如果调试模式已激活, 系统会捕捉遇到的 spy point (参见 `spy/1`) 和断点。此外, 最后调用优化会被禁用, 系统在销毁选择点时也会更保守, 以简化调试。

禁用这些优化可能导致在关闭调试模式时行为正确的程序耗尽内存。

debug_on_error (bool, rw)

如果为 `true`, 检测到错误后启动 tracer。否则继续执行。引发错误的目标通常会失败。另请参阅 Prolog 标志 `report_error`。默认值为 `true`。

debug_on_interrupt (bool, rw)

如果为 `true`, 在 Control-C 上启动调试器。更精确地说, 是在收到 SIGINT 时启动。初始值为 `false`, 进入交互式顶层时会设为 `true`。若要立即开始处理中断, 请参阅命令行选项 `--debug-on-interrupt`。

debugger_show_context (bool, rw)

如果为 `true`, tracer 打印栈帧时显示上下文模块。通常通过 tracer 的 `c` 选项控制。

debugger_write_options (term, rw)

该参数作为选项列表传给 `write_term/2`, 用于调试器打印目标。调试器的 `w`、`p` 和 `<N> d` 命令会修改它。默认值为 `[quoted(true), portray(true), max_depth(10), attributes(portray)]`。

determinism_error (atom, rw)

该标志定义谓词的不确定性与声明不一致时的行为。参见 `det/1`。可能值为 `error` (默认值)、`warning` 和 `silent`。

dialect (atom, r)

固定为 `swi`。下面的代码是检测 SWI-Prolog 的可靠且可移植方法：

```
is_dialect(swi) :-  
    catch(current_prolog_flag(dialect, swi), _, fail).
```

dir_sep (atom, r)

操作系统文件名中的目录分隔符。通常为 `/`，但 Windows 上为 `\`。

double_quotes (codes, chars, atom, string, rw)

该标志决定 Prolog 如何读取双引号字符串。和 `character_escapes`、`back_quotes` 一样，它按模块维护。默认值为 `string`，会生成字符串。如果给出 `--traditional`，默认值为 `codes`，会生成字符代码列表，即表示 Unicode 码点的整数。取值 `chars` 会生成单字符原子列表；取值 `atom` 会让双引号与单引号相同，从而创建原子。另请参阅扩展章节。

editor (atom, rw)

决定 `edit/1` 使用的编辑器。关于选择编辑器的细节，参见自定义编辑器章节。

emacs_inferior_process (bool, r)

如果为 `true`，SWI-Prolog 正作为 GNU/X-Emacs 的下级进程运行。如果环境变量 `EMACS` 为 `t` 且 `INFERIOR` 为 `yes`，SWI-Prolog 会假定这一点成立。

encoding (atom, rw)

以 `text` 模式打开文件时使用的默认编码。初始值从环境推导。详情参见编码章节。

executable (atom, r)

正在运行的可执行文件路径名。`qsave_program/2` 会把它作为默认仿真器。

executable_format (atom, r)

SWI-Prolog 可执行文件的格式，例如当 `swipl` 是 ELF 二进制文件时为 `elf`。

engines (bool, r)

如果支持 `engines`，则为 `true`。在多线程版本中总是如此。单线程版本的 SWI-Prolog 也可能启用 `engines`。

exit_status (integer, r)

由 `halt/1` 设置为其参数，使注册到 `at_halt/1` 的 `hook` 可以访问退出状态。

file_name_case_handling (atom, rw)

该标志定义 Prolog 如何处理文件名大小写。它用于大小写规范化，并用于判断两个名称是否指向同一文件。注意，文件名大小写处理通常是文件系统属性，而 Prolog 只有一个全局标志来决定其文件处理方式。该标志有下列取值：

- `case_sensitive`：文件系统完全区分大小写。Prolog 不执行任何大小写修改或不区分大小写的匹配。这是 Unix 系统上的默认值。
- `case_preserving`：文件系统不区分大小写，但保留用户创建文件时使用的大小写。这是 Windows 系统上的默认值。
- `case_insensitive`：文件系统既不存储也不匹配大小写。在这种情况下，Prolog 会把所有文件名映射为小写。

file_name_variables (bool, rw)

如果为 true (默认值为 false)，会在接受文件名的内建谓词参数中展开 \$varname 和 ~，例如 open/3、exists_file/1、access_file/2 等。谓词 expand_file_name/2 可用于展开环境变量和通配符模式。该 Prolog 标志旨在兼容旧版本 SWI-Prolog。

file_search_cache_time (number, rw)

absolute_file_name/3 搜索结果的缓存时间，单位为秒。在该时间限制内，系统会先检查旧搜索结果是否仍满足条件。默认值为 10 秒，通常可以避免编译期间对库文件等进行大多数重复搜索。将该值设为 0 会禁用缓存。

float_max (float, r)

可表示的最大浮点数。

float_max_integer (float, r)

能用浮点数精确表示的最大整数。

float_min (float, r)

大于 0.0 的最小可表示浮点数。另请参阅函数 nexttoward/2。

float_overflow (atom, rw)

取值为 error (默认值) 或 infinity。前者符合 ISO。使用 infinity 时，浮点溢出会映射为正或负 Inf。参见 IEEE 浮点章节。该标志也影响 read_term/3 及相关谓词，使它们把过大的浮点数读为 infinity。

float_rounding (atom, rw)

定义算术如何舍入为浮点数。已定义取值为 to_nearest (默认值)、to_positive、to_negative 或 to_zero。对大多数场景，函数 roundtoward/2 是更安全也更快的替代方案。

float_undefined (atom, rw)

取值为 error (默认值) 或 nan。前者符合 ISO。使用 nan 时，未定义操作 (例如 sqrt(-2.0)) 会映射为 NaN。参见 IEEE 浮点章节。

float_underflow (atom, rw)

取值为 error 或 ignore (默认值)。后者符合 ISO，会把结果绑定为 0.0。

float_zero_div (atom, rw)

取值为 error (默认值) 或 infinity。前者符合 ISO。使用 infinity 时，除以 0.0 会映射为正或负 Inf。参见 IEEE 浮点章节。

gc (bool, rw)

如果为 true (默认值)，垃圾回收器处于活动状态。如果为 false，则既不会执行垃圾回收，也不会执行栈移动，即使显式请求也不会执行。可以修改。

gc_thread (bool, r)

如果为 true (启用线程时默认如此)，原子和子句垃圾回收会在一个别名为 gc 的独立线程中执行。否则，由检测到足够垃圾的线程执行垃圾回收。由于运行这些全局回收器可能耗时较长，使用独立线程可以改善实时行为。可以使用 set_prolog_gc_thread/1 控制 gc 线程，该谓词要么启用 gc 线程，要么杀死 gc 线程并等待它结束。

generate_debug_info (bool, rw)

如果为 true (默认值), 生成可以用 trace/0、spy/1 等调试的代码。可以用 --no-debug 设为 false。该标志在源文件内有作用域。很多库使用 :- set_prolog_flag(generate_debug_info, false) 来在普通 trace 中隐藏其内部细节。在当前实现中, 这只会在此谓词上设置一个标志, 使子调用对调试器隐藏; 该名称预示编译器未来可能进一步变化。

gmp_version (integer, r)

如果 Prolog 链接了 GMP, 该标志给出所用 GMP 库的主版本。另请参阅 GMP 外部接口章节。链接到 LibBF 时不存在该标志。可通过 Prolog 标志 bounded 不存在来测试大整数和有理数支持。

gui (bool, r)

如果 XPC 存在且可用于图形界面, 则设置为 true。

halt_grace_time (float, rw)

halt/1 等待其他线程优雅结束的时间。默认值为 1 秒。

heartbeat (integer, rw)

如果非零, 则每 N 次推理调用一次 prolog:heartbeat/0。N 会四舍五入到 16 的倍数。

home (atom, r)

SWI-Prolog 所认为的 home 目录。SWI-Prolog 使用 home 目录查找启动文件 <home>/boot.prc, 并查找库目录 <home>/library。有些安装可能把体系结构无关文件放在共享 home 中, 并同时定义 shared_home。系统文件可通过 absolute_file_name/3 以 swi(file) 的形式找到。参见 file_search_path/2。关于该位置如何确定, 参见 home 查找章节; 关于从命令行设置或报告它, 参见 --home。

integer_rounding_function (down, toward_zero, r)

ISO Prolog 标志, 描述算术函数 // 和 rem 的舍入方式。取值取决于所用 C 编译器。

iso (bool, rw)

启用一些奇特的 ISO 兼容行为, 这些行为与正常 SWI-Prolog 行为不兼容。目前它有下列影响:

- 函子 //2 (浮点除法) 总是返回浮点数, 即使应用于可以整除的整数。
- 在项的标准序中, 所有浮点数都排在所有整数之前。
- 如果 atom_length/2 的第一个参数是数字, 则产生类型错误。
- 访问静态谓词时, clause/[2,3] 会引发权限错误。
- 访问静态谓词时, abolish/[1,2] 会引发权限错误。
- 语法更接近 ISO 标准: - 在函数式记法和列表记法中, 项的优先级必须低于 1000。这意味着作为参数出现的规则和控制结构需要加括号。像 [a :- b, c]. 这样的项现在必须消歧为 [(a :- b), c]. 或 [(a :- b, c)].。
- 作为操作数出现的运算符必须加括号。应写作 X == (-), true., 而不是 X == -, true.。目前这一点还没有完全强制执行。
- 反斜线转义的换行会按 ISO 标准解释。参见字符转义语法章节。

large_files (bool, r)

如果存在且为 true，表示 SWI-Prolog 编译时启用了大文件支持（large file support, LFS），可以访问大于 2GB 的文件。该标志在 64 位硬件上总是 true；在 32 位硬件上，如果配置检测到 LFS 支持，则为 true。注意，特定文件所在的文件系统仍可能限制文件大小。

last_call_optimisation (bool, rw)

决定是否启用最后调用优化。通常，该标志的值是 debug 标志的否定。由于省略最后调用优化可能让程序耗尽栈空间，有时需要在调试期间启用它。

libswipl (atom, rw)

SWI-Prolog 共享库 libswipl 所在路径，即提供 Prolog 的 SWI-Prolog 共享对象。在某些系统上，可以从正在运行的系统可靠确定该路径，这些系统上的标志是只读的。在其他系统上，它是配置的目标安装位置；如果安装被移动，该值可能错误。由于没有跨平台的可靠方式计算该路径，这些平台上的标志为读写。当前，该标志在 Windows 以及提供 dladdr() 函数的 POSIX 系统上可靠；Linux 和 MacOS 都提供该函数。

linux (bool, r)

如果存在且为 true，操作系统是某种 Linux。另请参阅 unix。

malloc (atom, r)

在成功识别所用 malloc() 实现后设置。当前可能取值为 tcmalloc 和 ptmalloc。详情参见内存分配章节。

max_answers_for_subgoal (integer, rw)

限制表中答案数量。原子 infinite 会清除该标志。默认情况下，该标志未定义。详情参见制表限制章节。

max_answers_for_subgoal_action (atom, rw)

当表达到 max_answers_for_subgoal 指定的答案数量时采取的动作。支持的取值为 bounded_rationality、error（默认值）或 suspend。

max_arity (unbounded, r)

ISO Prolog 标志，表示复合项没有最大元数限制。

max_char_code (integer, r)

支持的最高 Unicode 码点。SWI-Prolog 支持从 0 到该标志值（含）的所有 Unicode 码点。该值遵循 Unicode 标准，当前为 0x10ffff。

unicode_syntax_version (atom, r)

用于构建 SWI-Prolog 源语法分类器的数据的 Unicode 版本，例如 '17.0.0'。它驱动 Unicode Prolog 源语法章节中描述的 identifier、layout 和 solo 类。另请参阅 library(unicode) 中的 unicode_version/1，它报告绑定的 utf8proc 数据版本；该版本可能不同，用于规范化、字素分割和 unicode_property/2。

max_integer (integer, r)

如果整数是有界的，这是最大整数值。另请参阅标志 bounded 和算术类型章节。

max_integer_size(integer, rw)

设置这个 tripwire 后，为大整数和有理数分配内存时会限制为给定字节数。最小值为 1,000。未设置时，分配限制由栈限制决定，因为系统无法表示更大的数，也无法表示 malloc() 失败。特别是那些可能代表客户端处理任意算术表达式的服务，可以设置该限制以避免资源耗尽。

max_procedure_arity(integer, r)

谓词的最大元数。尝试定义或调用这样的谓词会产生 representation_error(max_procedure_arity) 异常。当前设为 1024。

max_rational_size(integer, rw)

限制有理数大小，单位为字节。这个 tripwire 可用于发现把 Prolog 标志 prefer_rationals 设为 true 后产生过大有理数的情况；如果不需要精度，应使用浮点算术。注意，有理数也会被 Prolog 标志 max_integer_size 隐式限制。

max_rational_size_action(atom, rw)

超过 max_rational_size tripwire 时采取的动作。可能值为 error（默认值），即抛出 tripwire 资源错误；以及 float，即把有理数转换为浮点数。注意，有理数可能超过浮点数范围。

max_table_answer_size(integer, rw)

限制制表中答案替换的大小。原子 infinite 会清除该标志。默认情况下，该标志未定义。详情参见制表限制章节。

max_table_answer_size_action(atom, rw)

如果向表中添加大于 max_table_answer_size 的答案替换，系统采取的动作。支持的取值为 error（默认值）、bounded_rationality、suspend 和 fail。

max_table_subgoal_size(integer, rw)

限制访问表的目标项大小。原子 infinite 会清除该标志。默认情况下，该标志未定义。详情参见制表限制章节。

max_table_subgoal_size_action(atom, rw)

如果制表目标超过 max_table_subgoal_size，系统采取的动作。支持的取值为 error（默认值）、abstract 和 suspend。

max_tagged_integer(integer, r)

可表示为 tagged 值的最大整数。Tagged 整数需要一个字的存储空间。更大的整数表示为间接数据（indirect data），需要显著更多空间。

message_context(list(atom), rw)

要添加到 error 和 warning 级别消息中的上下文信息。该列表可以包含元素 thread，用于把生成消息的线程加入消息；也可以包含 time 或 time(Format)，用于添加时间戳。默认时间格式为 %T.%3f。默认值为 [thread]。另请参阅 format_time/3 和 print_message/2。

min_integer(integer, r)

如果整数是有界的，这是最小整数值。另请参阅标志 bounded 和算术类型章节。

min_tagged_integer(integer, r)

Tagged integer 值范围的起点。

mitigate_spectre (bool, rw)

当为 true（默认值为 false）时，强制缓解基于时间的 Spectre 安全漏洞。基于 Spectre 的攻击可以从进程拥有但本应保持不可见的内存中提取信息，例如密码或 Web 服务器的私钥。这类攻击通过导致对敏感数据的推测性访问，并通过连续指令耗时差异等旁路泄漏数据。一个可能受影响的应用示例是 SWISH：它允许用户运行 Prolog 代码，而 SWISH 服务器必须同时保护其他用户的隐私以及 HTTPS 私钥、cookie 和密码。

目前，启用该标志会把 `get_time/1` 和 `statistics/2` CPU 时间的分辨率降低到 20 微秒。

警告：虽然更粗粒度的计时器会让这类攻击更难成功，但通常不能可靠防止此类攻击。完整缓解可能需要编译器支持，以禁用对敏感数据的推测性访问。

msys2 (bool, r)

如果存在，表示 SWI-Prolog 是在 MSYS2 shell 下运行的 MS-Windows 版本。

occurs_check (atom, rw)

该标志控制会创建无限树（也称循环项）的合一，并可取三个值。使用 false（默认值）时，合一成功并创建无限树。使用 true 时，合一行为类似 `unify_with_occurs_check/2`，静默失败。使用 error 时，尝试创建循环项会导致 `occurs_check` 异常。后者用于调试无意创建循环项的情况。注意，这是一个全局标志，会修改 Prolog 的基础行为。把该标志从默认值改掉，可能导致库无法正常工作。

on_error (atom, rw)

决定如何处理用 `print_message/2` 打印的错误，即报告给用户的错误。可能值为 `print`（默认值）、`status` 和 `halt`。使用 `halt` 时，进程立即以状态 1 停止。否则继续执行。使用 `status` 时，如果进程打印过一个或多个错误，`halt/0` 会以状态 1 退出。在编译模式（参见 `-c`）中，默认值为 `status`。该标志可通过命令行选项 `--on-error` 设置。另请参阅编译消息章节。

on_warning (atom, rw)

类似 `on_error`，但用于警告。默认值始终为 `print`。对应命令行选项为 `--on-warning`。

open_shared_object (bool, r)

如果为 true，`open_shared_object/2` 及相关谓词已实现，可访问共享库（.so 文件）或动态链接库（.DLL 文件）。

optimise (bool, rw)

如果为 true，以优化模式编译。若 Prolog 以命令行选项 `-O` 启动，初始值为 true。optimise 标志具有源文件作用域。

当前，优化编译意味着编译算术表达式，并删除可能由 `expand_goal/2` 产生的冗余 `true/0`。

未来版本可能包含其他优化，例如把小谓词集成到调用方、消除常量表达式和其他可预测结构。源代码优化从不应用于声明为动态的谓词（参见 `dynamic/1`）。

optimise_unify (bool, rw)

如果为 true（默认值），允许编译器移动或移除显式合一调用（`=/2`）。虽然这能显著提升性能，但源级调试器尚不能正确处理这种行为。参见函数体索引章节。

os_argv (list, rw)

一个原子列表，表示用于调用 SWI-Prolog 的命令行参数。注意，返回的列表包含所有参数。若要获取应用程序选项，请参阅 `argv`。

packs (bool, r)

如果为 true，扩展包 (add-ons) 已附加。可以使用 `--no-packs` 设为 false。

path_max (integer, r)

操作系统报告的文件路径名最大长度。这个长度通常不直接定义文件名字符数。实际限制可能因编码而更短；例如 POSIX 系统上，它通常定义经常为 UTF-8 编码的名称的长度限制。底层文件系统也可能施加额外限制。

path_sep (atom, r)

操作系统中文件搜索路径的分隔符，例如环境变量 PATH 使用的分隔符。通常为 `:`，但 Windows 上为 `;`。

pid (int, r)

正在运行的 Prolog 进程的进程标识符。该标志是否存在由实现定义。

pipe (bool, rw)

如果为 true，支持 `open(pipe(command), mode, Stream)` 等形式。可以修改，以便在测试该特性的应用程序中禁用 pipe。不推荐这样做。

portable_vmi (bool, rw)

如果为 true（默认值），生成可同时在 32 位和 64 位硬件上运行的 .qlf 文件和保存状态。如果为 false，某些优化的虚拟机指令只有在整数参数位于 32 位机器 tagged integer 范围内时才使用。

posix_shell (atom, rw)

POSIX 兼容 shell 的路径。默认通常为 `/bin/sh`。该标志由 `shell/1` 和 `qsave_program/2` 使用。

prefer_rationals (bool, rw)

只有在系统编译时支持无界和有理数算术时才提供（参见 `bounded`）。如果为 true，算术会优先产生有理数而非浮点数。这意味着：

- 两个整数相除（函数 `/2`）会产生有理数。
- 两个整数求幂（函数 `^/2`）会产生有理数，即使第二个操作数为负。例如，`2^(-2)` 求值得到 `1/4`。

使用 true 可能创建过大的有理数。Prolog 标志 `max_rational_size` 可用于检测并处理这个 tripwire。

如果为 false，有理数只能通过函数 `rational/1`、`rationalize/1`、`rdiv/2` 创建，或通过读取创建。另请参阅 `rational_syntax`、有理数语法章节和有理数章节。

当前默认值为 false。未来可能改为 true。强烈建议用户把该标志设为 true，并报告因此产生的问题。

print_write_options (term, rw)

指定 `print/1` 和 `print/2` 使用的 `write_term/2` 选项。

prompt_alternatives_on (atom, rw)

决定 Prolog 顶层如何提示替代答案。默认值为 `determinism`，表示如果目标成功但留下选择点，系统会提示替代答案。许多经典 Prolog 系统表现为 `groundness`：当且仅当查询包含变量时提示替代答案。

protect_static_code (bool, rw)

如果为 true (默认值为 false), clause/2 不操作静态代码, 从而对想列出 Prolog 程序静态代码的攻击者提供一些基本防护。一旦该标志为 true, 就不能改回 false。ISO 模式默认启用保护 (参见 Prolog 标志 iso)。注意, 开发环境的很多部分要求 clause/2 能操作静态代码, 因此启用该标志应只用于生产代码。

qcompile (atom, rw)

该选项为 load_files/2 的 qcompile(+Atom) 选项提供默认值。

rational_syntax (atom, rw)

决定有理数的读写语法。可能值为 natural (例如 1/3) 或 compatibility (例如 1r3)。compatibility 语法总是被接受。该标志对模块敏感。

当前默认值为 compatibility, 它会把有理数读写为例如 1r3。关于分隔字符仍有一些讨论, 参见有理数语法章节。未来可能考虑把默认值改为 natural。强烈建议用户把该标志设为 natural, 并报告因此产生的问题。

rationals (atom, r)

如果系统支持有理数, 则该标志存在且值为 true。对 SWI-Prolog 来说, 如果标志 bounded 为 false, 该标志总会设置。

readline (atom, rw)

指定提供哪种命令行编辑形式。可能值如下:

- false: 没有可用的命令行编辑。
- editline: 载入库 library(editline), 提供基于 BSD libedit 的行编辑。如果 library(editline) 可用, 这是默认值。

report_error (bool, rw)

如果为 true, 打印错误消息; 否则抑制它们。可以修改。另请参阅 Prolog 标志 debug_on_error。默认值为 true, 运行时版本除外。

resource_database (atom, r)

设置为附加状态的绝对文件名。通常是文件 boot32.prc、通过 -x 指定的文件, 或正在运行的可执行文件。另请参阅 resource/3。

runtime (bool, r)

如果存在且为 true, SWI-Prolog 以 -DO_RUNTIME 编译, 会禁用各种有用的开发功能 (目前包括 tracer 和 profiler)。

sandboxed_load (bool, rw)

如果为 true (默认值为 false), load_files/2 会调用 hook, 使 library(sandbox) 能验证指令的安全性。

saved_program (bool, r)

如果存在且为 true, 表示 Prolog 是从使用 qsave_program/[1,2] 保存的状态启动的。

shared_home (atom, r)

表示部分 SWI-Prolog 系统文件安装在 <prefix>/share/swipl, 而不是 home 下的 <prefix>/lib/swipl。该标志表示这个共享 home 的位置, 并且该目录会加入文件搜索路径 swi。参见 file_search_path/2 和标志 home。

shared_object_extension (atom, r)

操作系统用于共享对象的扩展名。多数 Unix 系统为 .so, Windows 为 .dll。它用于通过 file_type executable 定位文件。另请参阅 absolute_file_name/3。

shared_object_search_path (atom, r)

系统搜索共享对象时使用的环境变量名称。

shared_table_space (integer, rw)

为存储共享答案表保留的空间。参见共享制表章节和 Prolog 标志 table_space。

shift_check (bool, rw)

当为 true (默认值为 false) 时, 检查由 shift_for_copy/1 捕获的可疑定界延续。

signals (bool, r)

决定 Prolog 是否处理信号 (软件中断)。如果宿主操作系统不支持信号处理, 或命令行选项 --no-signals 处于活动状态, 该标志为 false。详情参见嵌入式信号章节。

source (bool, rw)

如果为 true, 在存在对应 .pl 文件时忽略 .qlf 文件。库中提供的 .qlf 文件是在启用优化 (参见 optimise)、启用宏展开 (参见 library(apply_macros)) 并移除 debug/3 和 assertion/1 语句的情况下编译的。使用该标志会载入源代码, 从而更好地支持调试。如果某个调试会话能更好地访问库调试设施中受益, 可以在程序载入文件开头设置该 Prolog 标志, 或这样启动 Prolog:

```
swipl -Dsource [option ...] myfile.pl ...
```

source_search_working_directory (bool, rw)

如果设为 true, 从源代码载入相对文件名时, 会同时相对源文件所在位置和工作目录搜索。相对工作目录搜索已弃用; 如果文件以这种方式找到, 会打印警告。未来版本可能把默认值改为 false。搜索工作目录一直支持到 9.3.8。9.3.9 禁用了该行为, 9.3.10 又带着警告重新启用了它。

stack_limit (int, rw)

限制当前线程的 Prolog 栈组合大小。另请参阅 --stack-limit 和内存限制章节。

stream_type_check (atom, rw)

定义系统是否以及多严格地验证: 字节 I/O 不应作用于文本流, 文本 I/O 不应作用于二进制流。取值为 false (不检查)、true (完整检查) 和 loose。使用 loose (默认值) 检查模式时, 系统接受从使用 ISO Latin-1 编码的文本流进行字节 I/O, 也接受向二进制流写入文本。

string_stack_tripwire (int, rw)

用于外部语言字符串管理的维护标志。如果字符串栈深度达到 tripwire 值, 则打印警告。详情参见外部接口字符串章节。

system_thread_id (int, r)

在多线程版本中可用, 前提是操作系统提供系统范围的整数线程标识符。该整数是操作系统用于调用线程的线程标识符。在 Linux 系统上, 这是该线程的 PID。

table_incremental (bool, rw)

设置是否使用增量制表的默认值。初始值为 `false`。参见 `table/1`。

table_shared (bool, rw)

设置是否使用共享制表的默认值。初始值为 `false`。参见 `table/1`。

table_space (integer, rw)

为存储制表谓词的答案表保留的空间（参见 `table/1`）。目前只计算答案 trie 中节点占用的空间。超过该空间时，会引发 `resource_error(table_space)` 异常。

table_subsumptive (bool, rw)

设置 variant 制表和 subsumptive 制表之间选择的默认值。初始值为 `false`。参见 `table/1`。

threads (bool, rw)

当支持线程时为 `true`。如果系统编译时没有线程支持，值为 `false` 且只读。否则，除非系统以 `--no-threads` 启动，值为 `true`。只有在线程尚未运行时才能禁用线程。另请参阅 `gc_thread` 标志。

timezone (integer, r)

当前时区相对 GMT 向西偏移的秒数。初始化时从与 POSIX `tzset()` 函数关联的 `timezone` 变量设置。另请参阅 `format_time/3`。

tmp_dir (atom, rw)

临时目录路径。从环境变量 `TMP` 或 `TEMP` 初始化。在 Windows 上使用这些变量；如果未定义，则使用默认值。默认值通常是 `/tmp`，Windows 上通常是 `c:/temp`。

toplevel_goal (term, rw)

定义运行初始化目标和入口点后执行的目标（参见 `-g`、`initialization/2` 和 `PrologScript` 章节）。初始值为 `default`，表示启动普通交互式会话。该值可以用命令行选项 `-t` 修改。显式值 `prolog` 等价于 `default`。如果使用 `initialization(Goal,main)` 且顶层为 `default`，顶层会设为 `halt`（参见 `halt/0`）。

toplevel_list_wfs_residual_program (bool, rw)

如果为 `true`（默认值），且答案根据良基语义（Well Founded Semantics, WFS）为 `undefined`，则在答案前列出 residual program。否则答案以 **undefined** 终止。另请参阅 `undefined/0`。

toplevel_mode (atom, rw)

如果为 `backtracking`（默认值），顶层会在完成查询后回溯。如果为 `recursive`，顶层实现为递归循环。这意味着使用 `b_setval/2` 设置的全局变量会在查询之间保持。在 `recursive` 模式下，顶层变量的答案（参见“复用顶层绑定”）保存在可回溯全局变量中，因此**不会被复制**。在 `backtracking` 模式下，顶层变量答案保存在 `recorded database` 中。

递归模式是为交互式使用 `CHR` 添加的，因为 `CHR` 会把全局约束存储保存在可回溯全局变量中。该建议来自 Falco Nogatz。

toplevel_name_variables (bool, rw)

如果为 `true`（默认值），在顶层为变量命名，而不是把它们打印为 `_NNN`。变量会命名为 `_A`、`_B` 等。只出现一次的变量（singleton）会打印为 `_`。

toplevel_print_anon (bool, rw)

如果为 true，以下划线 (_) 开头的顶层变量会正常打印。如果为 false (默认值)，这类变量的绑定会从答案中省略。可用于在复杂查询中从顶层隐藏绑定。例如，下面 _List 的绑定不会打印：

```
?- numlist(1,1 000 000,_List), sum_list(_List, Sum).
Sum = 500000500000.
```

toplevel_print_factorized (bool, rw)

如果为 true (默认值为 false)，显示答案替换中子项的内部共享。下面的示例揭示了由 library(rbtrees) 谓词 rb_new/1 实现的红黑树中叶节点的内部共享：

```
?- set_prolog_flag(toplevel_print_factorized, true).
?- rb_new(X).
X = t(_S1, _S1), % where
    _S1 = black(' ', _G387, _G388, '').
```

如果该标志为 false，% where 记法仍用于表示循环，如下例所示。该示例还显示，实现揭示的是内部循环长度，而不是最小循环长度。在 Prolog 中，不同长度的循环无法区分，S == R 就说明了这一点。

```
?- S = s(S), R = s(s(R)), S == R.
S = s(S),
R = s(s(R)).
```

toplevel_prompt (atom, rw)

定义交互式顶层使用的提示符。下面的 ~ (tilde) 序列会被替换：

序列	替换内容
~m	如果不是 user，替换为输入模块 (type-in module, 参见 module/1)
~l	如果不为 0，替换为 break level (参见 break/0)
~d	如果不是普通执行，替换为调试状态 (参见 debug/0、trace/0)
~!	如果启用了历史，替换为历史事件 (参见标志 history)

toplevel_residue_vars (bool, rw)

当为 true (默认值为 false) 时，打印由 call_residue_vars/2 检测到、但未出现在目标返回绑定中的 residual variables。

toplevel_thread (bool, rw)

当为 true 时，该线程正在运行顶层 REPL 循环。参见 prolog/0。

toplevel_var_size (int, rw)

在顶层查询中作为变量绑定返回、并保存下来以便通过 \$ 变量引用复用的项，其最大大小 (按 literal 计)。当为 0 时，变量记录和复用被禁用。参见“复用顶层绑定”。

trace_gc (bool, rw)

如果为 true (默认值为 false)，垃圾回收和栈移动会在终端上报告。可以修改。值以字节为单位报告为 G+T，其中 G 是全局栈值，T 是 trail 栈值。Gained 描述回收的字节数。used 是 GC 后栈上使用的字节数；free 是已分配但未使用的字节数。下面是输出示例：

```
% GC: gained 236,416+163,424 in 0.00 sec;
      used 13,448+5,808; free 72,568+47,440
```

traditional (bool, r)

在 SWI-Prolog 7 中可用。如果为 true，表示使用 --traditional 选择了 traditional 模式。注意，某些 SWI7 特性（例如 dict 上的函数式记法）在该模式下不可用。另请参阅扩展章节。

tty_control (bool, rw)

决定终端是否切换到 raw 模式以支持 get_single_char/1，该谓词也读取 trace 中的用户操作。可以设置。如果该标志在启动时为 false，命令行编辑会被禁用。另请参阅命令行选项 --no-tty。

unicode_atoms (atom, rw)

新打开文本流的默认原子内容策略；可通过 set_stream/2 按流覆盖，也可通过 open/4 的 unicode_atoms 选项覆盖，还可通过 read_term/2,3 和 read_clause/2,3 的 unicode_atoms 选项按调用覆盖。四个取值为 accept、nfc、error、reject，其效果见 read_term/2,3。默认值为 accept。

把该标志设为 nfc 时，如果尚未注册内核规范化 hook，会自动载入 library(unicode)；如果该库不可用，set_prolog_flag/2 调用会传播 use_module/1 引发的 existence_error(source_sink, library(unicode))。模式 error 不需要该 hook：当 atom_normalize_hook 为 false 时，它会回退到基于 wcwidth 的检查，把任何 wcwidth 小于 1 的码点（组合标记、零宽和不可打印字符）视为非 NFC。这可能过度拒绝泰语等在 NFC 中使用组合标记的文字系统。

不管该标志如何，SWI-Prolog 读取器都会无条件拒绝 Unicode bidi override 和 isolate 码点 (U+202A 到 U+202E，以及 U+2066 到 U+2069)，如果它们作为原始字节出现在未加引号的原子、运算符、变量、带引号的原子、字符串或注释中。这缓解了“Trojan source”攻击 (CVE-2021-42574)。如果程序需要在字面量原子或字符串中使用这类码点，必须使用相应的转义序列，例如 \u202E。

unix (bool, r)

如果存在且为 true，操作系统是某种 Unix。如果用于编译该 SWI-Prolog 版本的 C 编译器定义了 __unix__ 或 unix，则定义该标志。其他系统上不可用。另请参阅 linux、apple 和 windows。

unknown (fail,warning,error, rw)

决定遇到未定义过程时的行为。如果为 fail，谓词静默失败。如果为 warn，打印警告，并像谓词未定义一样继续执行。如果为 error (默认值)，引发 existence_error 异常。该标志局部于每个模块，并从模块的 import-module 继承。使用默认设置时，这意味着普通模块从 user 继承该标志，而 user 又从 system 继承值 error。用户可以修改模块 user 的标志，从而改变所有应用程序模块的默认值，也可以修改特定模块的值。强烈建议保留 error 默认值，并使用 dynamic/1 和/或 multifile/1 指定谓词可能不存在。

unknown_option(ignore,warning,error,rw)

决定处理选项列表的谓词收到不认识的选项时的行为。ISO 标准要求引发 `domain_error` 异常。但这被认为不实用：如果不同 Prolog 系统支持不同选项，它会让编写可移植代码变得困难；也会让那些处理选项并把一部分选项传给一个谓词、另一部分选项传给另一个谓词的谓词难以编写。例如，一个把文件读入项列表的谓词必须把选项分发给 `open/4` 和 `read_term/3`。除 ISO 模式外，SWI-Prolog 一直忽略未知选项（参见 `iso` 标志）。该标志提供对选项处理方式的完整控制。

unload_foreign_libraries(bool, rw)

如果为 `true`（默认值为 `false`），卸载所有已载入的外部库。默认值为 `false`，因为现代操作系统无论如何都会回收资源，而且卸载外部代码可能导致已注册的 `hook` 指向不再存在的数据或代码。

user_flags(Atom, rw)

定义 `set_prolog_flag/2` 在标志未知时的行为。取值为 `silent`、`warning` 和 `error`。前两个取值会即时创建标志，其中 `warning` 会打印消息。取值 `error` 与 ISO 一致：它引发存在性错误，并且不创建标志。另请参阅 `create_prolog_flag/3`。默认值为 `silent`，但未来版本可能改变这一点。鼓励开发者使用其他值，并确保为库使用 `create_prolog_flag/3` 正确创建标志。

var_prefix(bool, rw)

如果为 `true`（默认值为 `false`），变量必须以下划线（`_`）开头。可以修改。该标志局部于修改它的模块。参见变量前缀章节。

var_tag(Atom, rw)

该标志控制当 `Tag{...}` 中的 `Tag` 未绑定时如何解释。可能值为：

- `dict`：读作带未绑定 `tag` 的 `dict`（参见双向 `dict` 章节）。这是当前默认值。使用未绑定 `tag` 已弃用。适当时候默认值会改变，最终改为 `attvar`。参见 `dict` 兼容性章节。
- `attvar`：把 `Var{...}` 读作 `attributed variable`。这很可能成为未来默认值。
- `#`：把 `Var{...}` 读作 `#{...}`。该标志允许快速评估使用 `#{...}` 而不是带未绑定 `tag` 的 `dict` 会对代码造成什么影响。
- `warning`：类似 `#`，但打印警告。
- `error`：把 `Var{...}` 视为语法错误。

verbose(atom, rw)

该标志由 `print_message/2` 使用。如果值为 `silent`，类型为 `informational` 和 `banner` 的消息会被抑制。命令行选项 `-q` 会把初始值 `normal` 切换为 `silent`。

verbose_autoload(bool, rw)

如果为 `true`，自动加载库时会打印普通 `consult` 消息。默认会抑制该消息。该标志用于调试。

verbose_file_search(bool, rw)

如果为 `true`（默认值为 `false`），打印消息说明 `absolute_file_name/[2,3]` 定位文件的进展。用于调试复杂文件搜索路径。另请参阅 `file_search_path/2`。

verbose_load(atom, rw)

决定载入（编译）Prolog 文件时打印哪些消息。当前取值为 `full`（每个文件载入开始和结束时打印消息）、`normal`（每个文件载入结束时打印消息）、`brief`（顶层文件载入结束时打印消

息) 和 `silent` (不打印消息, 默认值)。该标志的值通常由 `load_files/2` 提供的 `silent(Bool)` 选项控制。

version(integer, r)

版本标识符是一个整数, 其值为:

```
10000 * Major + 100 * Minor + Patch
```

version_data(swi(Major, Minor, Patch, Extra), r)

方言兼容层的一部分; 另请参阅 Prolog 标志 `dialect` 和方言章节。Extra 以列表形式提供平台特定版本信息。Extra 用于 “7.4.0-rc1” 这样的 tagged version, 在这种情况下, Extra 包含项 `tag(rc1)`。

version_git(atom, r)

如果系统是从 git 仓库创建的, 则可用。详情参见 `git-describe`。

vmi_builtin(bool, rw)

决定 `true/0`、`atom/1` 等知名内建谓词是否通过转换为虚拟机代码来处理。除非启用调试模式, 该标志默认值为 `true`。把它设为 `false` 可能改善其他运行时 instrumentation 的结果。注意, 优化算术 (`-0`, 参见 Prolog 标志 `optimise`) 目前不会转换为普通谓词调用。

warn_autoload(bool, rw)

如果为 `true` (默认值为 `false`), 当从定义全局项展开或目标展开规则的文件自动加载谓词时发出警告。这些规则通常能提升性能或提供更清晰语义, 因此不建议自动加载。未来版本将默认启用该标志。

warn_override_implicit_import(bool, rw)

如果为 `true` (默认值), 当隐式导入的谓词被本地定义覆盖时打印警告。详情参见 `use_module/1`。

win_file_access_check(atom, rw)

控制 Windows 下 `access_file/2` 的行为。在 Windows 上没有可靠方法检查文件和目录访问权限。该标志允许在三种近似方案之间切换:

- `access`: 使用 `Windows_waccess()` 函数。它会忽略 ACL (Access Control List), 因此可能在实际不允许访问时指示访问被允许。
- `getfilesecurity`: 使用 `Windows_GetFileSecurity()` 函数。它并非在所有文件系统上都可用, 但在支持它的文件系统上可能是最佳选择, 尤其是本地 NTFS 卷。
- `openclose`: 尝试打开并关闭文件。这对文件可靠, 但对目录不可靠。目前目录用 `_waccess()` 检查。这是默认值。

windows(bool, r)

如果存在且为 `true`, 操作系统是 Microsoft Windows 的某种实现。该标志只在基于 MS-Windows 的版本上可用。另请参阅 `unix`。

wine_version(atom, r)

如果存在, 表示 SWI-Prolog 是在 Wine 仿真器下运行的 MS-Windows 版本。

write_attributes (atom, rw)

定义 write/1 及相关谓词如何写出 attributed variables。选项值随 write_term/2 的 attributes 选项一起说明。默认值为 ignore。

write_help_with_overstrike (bool, r)

help/1 写入终端时使用的内部标志。如果存在且为 true，它会使用 overstrike 打印粗体和下划线文本。

xdg (bool, r(w))

该标志定义是否遵循 Free Desktop 标准来处理应用程序数据和配置文件。非 Windows 系统上，该标志为 true 且只读。在 Windows 上，如果是在 Conda 或 MSYS2 下编译，该标志为 true 但可读写；否则未定义。在 Windows 上，搜索顺序如下：

- 标志未定义：先搜索 Windows 目录，再搜索 XDG 目录。这是 Windows 二进制包的默认值。
- 标志为 true：只搜索 XDG 目录。
- 标志为 false：只搜索 Windows 目录。

xpce (bool, r)

如果 XPCE 图形系统已载入，则可用并设置为 true。

xpce_version (atom, r)

如果 XPCE 系统已载入，则可用并设置为其版本。

xref (bool, rw)

如果为 true，源代码是为了分析目的读取，例如交叉引用。否则（默认值）源代码是为了编译而读取。该标志在若干地方由 term_expansion/2 和 goal_expansion/2 hook 使用，尤其是这些 hook 带有副作用时。另请参阅库 library(prolog_source) 和 library(prolog_xref)。

set_prolog_flag/2

set_prolog_flag(:Key, +Value) 设置一个 Prolog 标志的值。Key 是原子。如果该标志是系统定义的标志，且未在上文标记为可变，尝试修改它会产生 permission_error。如果提供的 Value 与该标志类型不匹配，会引发 type_error。

有些标志（例如 unknown）按模块维护。目标模块由 Key 这个 meta 参数决定。

除了 ISO 规定的行为外，SWI-Prolog 允许用户定义 Prolog 标志。新的 Prolog 标志应使用 create_prolog_flag/3 创建。出于历史原因，如果 Prolog 标志 user_flags 为 true（默认值），set_prolog_flag/2 会静默创建 Prolog 标志；也就是说，set_prolog_flag/2 的行为类似下面这样：

```
set_prolog_flag(Key, Value) :-
    current_prolog_flag(Key, _),
    !,
    <set the flag>.
set_prolog_flag(Key, Value) :-
    current_prolog_flag(user_flags, true),
    !,
    create_prolog_flag(Key, Value, []).
set_prolog_flag(Key, _) :-
    existence_error(prolog_flag, Key).
```

create_prolog_flag/3

`create_prolog_flag(+Key, +Value, +Options)` 创建新的 Prolog 标志。ISO 标准没有预见新标志的创建，但许多库都会引入新标志。在多线程环境中，Prolog 标志特别适合管理会变化的全局设置。谓词要么局部于某个线程，要么在所有线程之间共享；而线程会从创建它的线程继承标志（参见 `thread_create/3`），之后的修改则局部于调用线程。

SWI-Prolog 标志有类型。如果没有用 `type(Type)` 选项显式定义类型，类型会从初始值确定。已定义类型包括：`boolean`（如果初始值是 `false`、`true`、`on` 或 `off` 之一）、`atom`（如果初始值是其原子）、`integer`（如果该值是可表示为 64 位有符号值的整数）。任何其他初始值都会产生无类型标志，可以表示任何有效 Prolog 项。

默认情况下，新标志会加入全局标志表，使尚未设置该标志的所有线程都能访问该值。如果该标志已在调用线程中局部定义，则调用线程和全局标志表中的值都会更新。参见 `local(+Boolean)` 选项。

`Options` 是下列选项的列表。另请参阅 Prolog 标志 `user_flags`。

- `access(+Access)`：定义该标志的访问权限。取值为 `read_write` 和 `read_only`。默认值为 `read_write`。
- `type(+Atom)`：定义类型限制。可能值为 `boolean`、`atom`、`oneof(ListOfAtoms)`、`integer`、`float` 和 `term`。默认值由初始值决定。注意，`term` 会把项限制为 `ground`。
- `keep(+Boolean)`：如果为 `true`，当标志已存在时不修改它。否则（默认值），如果标志已存在，该谓词行为类似 `set_prolog_flag/2`。

如果标志已有值，但该值与指定类型不兼容，系统会打印警告，并把标志设置为本次 `create_prolog_flag/3` 调用指定的值和类型。

- `local(+Boolean)`：如果为 `true`（默认值为 `false`），并且该标志不存在，则只在调用线程中创建它。该标志只对调用线程以及从调用线程继承的线程可见。
- `warn_not_accessed(+Boolean)`：如果为 `true`，且该标志从未用 `current_prolog_flag/2` 读取，则打印警告。该选项用于通过命令行选项 `-D<flag>[=<value>]` 设置的选项。

push_prolog_flag/2

`push_prolog_flag(:Key, +Value)` 保存当前线程局部的 `Key` 值，并把它设置为 `Value`。如果 `Key` 不存在，则创建它（仅在调用线程中），并且被压入的状态会记录其原本不存在。该操作可嵌套，并与 `pop_prolog_flag/1` 配对。

这些谓词主要面向两个用例。第一个是使用不同标志进行有作用域的编译。例如，下面的代码会保留子句的书写形式。如果不局部修改该标志，`X = 42` 通常会被移入头部。

```
:- push_prolog_flag(optimise_unify, false).
p(X) :-
    X = 42,
    format('The answer to the ultimate question~n').
:- pop_prolog_flag(optimise_unify).
```

第二个用例是运行时作用域。例如，执行一个带 `occurs check` 的目标：

```
call_with_occurs_check(Goal) :-
    setup_call_cleanup(
```

```
push_prolog_flag(occurs_check, true),  
Goal,  
pop_prolog_flag(occurs_check)).
```

注意，pop 会在 Goal 完成时发生。尤其是当 Goal 成功但留下选择点时，它不会立即执行。

pop_prolog_flag/1

pop_prolog_flag(:Key) 恢复与之匹配的 push_prolog_flag/2 保存的状态。如果在匹配的 push 时 Key 不存在，则再次移除它。如果当前线程上不存在匹配的 push，则引发 existence_error(pushed_flag, Key)。

钩子谓词概览

SWI-Prolog 提供了大量 hook，主要用于控制消息处理、调试、启动、关闭、宏展开等行为。下面汇总了所有已定义的 hook，并标明其可移植性。

- `portray/1`

接入 `write_term/3`，修改项的打印方式（ISO）。

- `message_hook/3`

接入 `print_message/2`，修改系统消息的打印方式（Quintus/SICStus）。

- `message_property/2`

接入 `print_message/2`，定义前缀、输出流、颜色等属性。

- `message_prefix_hook/2`

接入 `print_message/2`，向消息添加额外前缀，例如时间和线程。

- `library_directory/1`

接入 `absolute_file_name/3`，定义新的库目录（多数 Prolog 系统）。

- `file_search_path/2`

接入 `absolute_file_name/3`，定义新的搜索路径（Quintus/SICStus）。

- `term_expansion/2`

接入 `load_files/2`，在读取到的项被编译前修改它们，也就是进行宏处理（多数 Prolog 系统）。

- `goal_expansion/2`

类似 `term_expansion/2`，但作用于单个目标（SICStus）。

- `prolog_load_file/2`

接入 `load_files/2`，从“非文件”资源中把其他数据格式载入为 Prolog 源。谓词 `load_files/2` 是 `consult/1`、`use_module/1` 等谓词的祖先。

- `prolog_edit:locate/3`

接入 `edit/1`，定位对象（SWI）。

- `prolog_edit:edit_source/1`

接入 `edit/1`，调用内部编辑器（SWI）。

- `prolog_edit:edit_command/2`

接入 `edit/1`，定义要使用的外部编辑器（SWI）。

- `prolog_list_goal/1`

接入 `tracer`，列出与特定目标关联的代码（SWI）。

- `prolog_trace_interception/4`

接入 `tracer`，处理 `trace` 事件（SWI）。

- `prolog:debug_control_hook/1`

SWI-Prolog 中文文档

接入 spy/1、nospy/1、nospyall/0 和 debugging/0，把这些控制谓词扩展到更高层的库。

- prolog:help_hook/1

接入 help/0、help/1 和 apropos/1，扩展帮助系统。

- resource/3

定义新的资源。严格来说这不是真正的 hook，但与 hook 类似 (SWI)。

- exception/3

早期对通用 hook 机制的一次尝试。它处理未定义谓词 (SWI)。

- attr_unify_hook/2

attributed variable 的合一 hook。可以在任何模块中定义。详情参见 attributed variable 章节。

库的自动加载

如果在运行时捕获到未定义谓词，系统会先尝试从该模块的默认模块导入该谓词（参见 `import module` 相关章节）。如果失败，自动加载器（auto loader）会被激活。实际过程是先调用 `hook user:exception/3`；只有该 `hook` 失败时，才会调用自动加载器。

第一次激活时，系统会把所有库目录中所有库文件的索引载入内存（参见 `library_directory/1`、`file_search_path/2` 和 `reload_library_index/0`）。如果能在某个库中找到该未定义谓词，对应库文件会被自动载入，并重新启动对这个此前未定义谓词的调用。默认情况下，该机制会静默载入文件。`current_prolog_flag/2` 的键 `verbose_autoload` 可用于获得详细载入信息。Prolog 标志 `autoload` 可用于启用或禁用自动加载系统。`autoload/[1,2]` 提供了更受控的自动加载形式，也支持应用模块的惰性加载。

自动加载只处理使用模块机制的库源文件。文件会用 `use_module/2` 载入，并且只有被捕获的未定义谓词会被导入到调用该未定义谓词的模块中。每个库目录都必须包含一个 `INDEX.pl` 文件，其中包含该目录所有库文件的索引。该文件由如下格式的行组成：

```
index(Name, Arity, Module, File).
```

谓词 `make/0` 会更新自动加载索引。它会搜索所有库目录（参见 `library_directory/1` 和 `file_search_path/2`），查找包含 `MKINDEX.pl` 或 `INDEX.pl` 的目录。如果当前用户可以写入或创建 `INDEX.pl`，并且该文件不存在、或早于该目录或其中某个文件，则更新该目录的索引。如果 `MKINDEX.pl` 存在，则通过载入该文件来更新索引；该文件通常包含一个调用 `make_library_index/2` 的指令。否则会调用 `make_library_index/1`，为所有包含模块的 `*.pl` 文件创建索引。

下面是一个创建已索引库目录的示例：

```
% mkdir ~/${XDG_DATA_HOME-.config}/swi-prolog/lib
% cd ~/${XDG_DATA_HOME-.config}/swi-prolog/lib
% swipl -g 'make_library_index(.)' -t halt
```

如果有多个库文件包含所需谓词，则按下面的搜索方案查找：

1. 如果某个库文件定义了捕获未定义谓词的模块，则使用该文件。
2. 否则，按 `library_directory/1` 谓词中出现的顺序考虑库文件；在同一目录内按字母顺序考虑。

autoload_path/1

`autoload_path(+DirAlias)` 把 `DirAlias` 添加到自动加载器使用的库中。这会扩展搜索路径 `autoload`，并重新载入库索引。例如：

```
:- autoload_path(library(http)).
```

如果该调用作为指令出现，它会被项展开为一个 `user:file_search_path/2` 子句，以及一个调用 `reload_library_index/0` 的指令。这样可以保留源代码信息，并允许移除该指令。

make_library_index/1

`make_library_index(+Directory)` 为该目录创建索引。索引会写入指定目录中的 `INDEX.pl` 文件。如果目录不存在或写保护，则失败并给出警告。

make_library_index/2

`make_library_index(+Directory, +ListOfPatterns)` 通常在 `MKINDEX.pl` 中使用。该谓词会为 `Directory` 创建 `INDEX.pl`，为匹配 `ListOfPatterns` 中任一文件模式的所有文件建立索引。

有时库包包含一个公共载入文件，以及若干由该载入文件使用的文件；这些内部文件导出的谓词不应由最终用户直接使用。这样的库可以放在库的子目录中，而包含公共功能的文件可以加入该库的索引。下面以 `XPCE` 库的 `MKINDEX.pl` 为例，它把 `trace/browse.pl` 的公共功能加入 `XPCE` 包可自动加载的谓词中。

```
:- prolog_load_context(directory, Dir),
   make_library_index(Dir,
                      [ '*.pl',
                        'trace/browse.pl',
                        'swi/*.pl'
                      ]).
```

reload_library_index/0

修改库目录集合后，可用 `reload_library_index/0` 强制重新载入索引。修改库目录集合的方式包括：更改 `library_directory/1`、`file_search_path/2` 的规则，添加或删除 `INDEX.pl` 文件。该谓词**不会**更新 `INDEX.pl` 文件。若要更新索引文件，请查看 `make_library_index/[1,2]` 和 `make/0`。

通常，如果某个谓词无法在索引中找到，并且库目录集合已经改变，索引会自动重新载入。如果目录被移除，或库目录顺序发生改变，则必须使用 `reload_library_index/0`。

使用 `qsave_program/2` 或命令行选项 `-c` 创建可执行文件时，必须显式载入所有通常会自动加载的谓词。这一点在运行时章节中讨论。另请参阅 `autoload_all/0`。

SWI-Prolog 语法

SWI-Prolog 的语法接近 ISO Prolog 标准语法，而 ISO Prolog 又基于 Edinburgh Prolog 语法。形式化描述可以在 ISO 标准文档中找到。若需要非正式介绍，可参考 Prolog 教科书以及在线教程。除了这里记录的 ISO 标准差异之外，SWI-Prolog 还提供了若干扩展，其中一部分也扩展了语法。更多信息见扩展相关章节。

ISO 语法支持

本节列出 SWI-Prolog 相对于 ISO Prolog 语法的若干扩展。

处理器字符集

处理器字符集规定解析 Prolog 源文本时每个字符所属的类别。字符分类固定为 Unicode。另见宽字符支持相关章节。

嵌套注释

SWI-Prolog 允许嵌套 `/* ... */` 注释。ISO 标准会把 `/* ... /* ... */` 接受为一段注释，而 SWI-Prolog 会继续寻找终止用的 `*/`。如果要把一段本身已经包含 `/* ... */` 注释的代码整体注释掉，这会很有用。这个修改也避免了下面示例中的意外注释问题：第一段注释的结束 `*/` 被忘记了。

```
/* comment

code

/* second comment */

code
```

字符转义语法

在带引号的原子（使用单引号：'`<atom>`'）中，特殊字符用转义序列表示。转义序列由反斜杠（`\`）引入。转义序列列表兼容 ISO 标准，但包含一些扩展；为提升兼容性，数值指定字符的解释也略微更灵活。未定义的转义字符会抛出 `syntax_error` 异常。SWI-Prolog 6.1.9 及以前版本会逐字复制未定义转义字符，即去掉反斜杠。

转义	含义
<code>\a</code>	警告字符。通常是 ASCII 字符 7（响铃）。
<code>\b</code>	退格字符。
<code>\c</code>	不产生输出。跳过输入中直到第一个非布局字符之前的所有字符。这可用于写出更美观的长行。ISO 不支持。
<code>\</code> 后接换行	在 ISO 模式中（见 Prolog 标志 <code>iso</code> ），只跳过这个序列。在原生模式中，换行后的空白也会被跳过，并打印警告，指出该结构已弃用并建议使用 <code>\c</code> 。
<code>\e</code>	转义字符（ASCII 27）。非 ISO，但被广泛支持。
<code>\f</code>	换页字符。

<code>\n</code>	下一行字符。
<code>\r</code>	仅回车，即回到行首。
<code>\s</code>	空格字符。用于允许写出 <code>0'\s</code> 来取得空格字符的字符码。非 ISO。
<code>\t</code>	水平制表符。
<code>\v</code>	垂直制表符（ASCII 11）。
<code>\xXX.. \</code>	字符的十六进制表示。按 ISO 标准，结尾的 <code>\</code> 是必需的；但 SWI-Prolog 中它是可选的，以增强与较早 Edinburgh 标准的兼容性。代码 <code>\xa3</code> 产生字符 10（十六进制 a）后接 3。这种方式指定的字符按 Unicode 字符解释。另见 <code>\u</code> 。
<code>\uXXXX</code>	Unicode 字符表示，其中字符用 恰好 4 个十六进制数字指定。这是 ISO 标准的扩展，修复了两个问题：首先， <code>\x</code> 定义的是数字字符码，但没有指定该字符码应在哪个字符集中解释；其次，不再需要 ISO Prolog 中特别的结尾反斜杠语法。
<code>\UXXXXXXXX</code>	与 <code>\uXXXX</code> 相同，但使用 8 个数字，以覆盖整个 Unicode 集。
<code>\40</code>	八进制字符表示。十六进制表示的规则和说明同样适用于八进制表示。
<code>\\</code>	转义反斜杠本身。因此， <code>'\\'</code> 是一个只包含单个 <code>\</code> 的原子。
<code>\'</code>	单引号。注意， <code>'\''</code> 和 <code>''''</code> 都描述只包含单个 <code>'</code> 的原子，也就是说 <code>'\'' == ''''</code> 为真。
<code>\"</code>	双引号。
<code>\`</code>	反引号。

`\c` 的示例：

```
format('This is a long line that looks better if it was \c
      split across multiple physical lines in the input')
```

对于反斜杠后接换行的长行写法，建议使用 `\c`，或者把布局字符放在 `\` 之前，如下所示。`\c` 被多个其他 Prolog 实现支持，并将继续被 SWI-Prolog 支持。下面这种风格是兼容性最好的方案。

```
format('This is a long line that looks better if it was \
      split across multiple physical lines in the input')
```

而不是：

```
format('This is a long line that looks better if it was\
split across multiple physical lines in the input')
```

注意，SWI-Prolog 也允许未转义的换行出现在带引号的材料中。ISO 标准不允许这样做，但过去这曾是常见实践。

只有在 `current_prolog_flag(character_escapes, true)` 生效时，字符转义才可用（默认如此）。见 `current_prolog_flag/2`。字符转义会以两种方式与 `writeln/2` 冲突：`\40` 会被 `writeln/2` 解释为十进制 40，但会被 `read` 解释为八进制 40（十进制 32）。此外，`writeln/2` 序列 `\l` 是非法的。建议改用支持更广泛的 `format/[2,3]` 谓词。如果坚持使用 `writeln/2`，可以把 Prolog 标志 `character_escapes` 切换为 `false`，也可以使用双重 `\\`，例如 `writeln('\\l')`。

非十进制数的语法

SWI-Prolog 同时实现 Edinburgh 和 ISO 两种非十进制数表示。按照 Edinburgh 语法，这类数写作 `<radix>'<number>`，其中 `<radix>` 是 2 到 36 之间的数字。ISO 使用 `0[bxo]<number>` 定义二进制、八进制和十六进制数。例如，下面是一个合法表达式：

```
A is 0b100 \\ 0xf00
```

这类数总是无符号的。

在大整数中使用数字分组

SWI-Prolog 支持把长整数拆分成数字分组（digit groups）。数字分组可以用“下划线 + 可选空白”的序列分隔。如果基数小于或等于 10，也可以用恰好一个空格分隔。下面几种写法都表示整数 100 万：

```
1_000_000
1 000 000
1_000_/*more*/000
```

可以用 `format/2` 和格式说明符 `~I` 按这种记法打印整数。例如：

```
?- format('~I', [1000000]).
1_000_000
```

当前语法由 Ulrich Neumerkel 在 SWI-Prolog 邮件列表上提出。

有理数语法

从 8.1.22 版开始，如果 SWI-Prolog 在编译时启用了 GMP 库，它会把有理数作为一等原子数据类型支持。这可以用 Prolog 标志 `bounded` 测试。原子类型也需要一种语法。遗憾的是，在不破坏 ISO 标准的前提下加入有理数，选择并不多。ECLiPSe 使用 `numerator_denominator`。该语法与 SWI-Prolog 的数字分组冲突（见上一节），而且没有被广泛认可为有理数写法。`1/3r` 和 `1/3R` 也曾被提出；`1/3r` 与 Ruby 兼容，但因为需要向前看而难以解析，也不太自然。另见 https://en.wikipedia.org/wiki/Rational_data_type。

ECLiPSe 和 SWI-Prolog 已同意把有理数的规范语法定义为类似 `1r3` 的形式。此外，ECLiPSe 接受 `1_3`；SWI-Prolog 可通过模块敏感的 Prolog 标志 `rational_syntax` 接受 `1/3`。该标志的取值如下。注意，`write_canonical/1` 始终使用兼容的 `1r3` 语法。

- **natural**：默认模式。在该模式下，系统忽略歧义问题，并采用最自然的 `<integer>/<nonneg>` 形式。这里，`<integer>` 遵循 Prolog 十进制整数的常规规则；`<nonneg>` 也遵循同样规则，但不允许符号。解析器会把有理数转换为规范形式，这意味着结果的分子和分母没有公因子。有理数示例：

输入	规范形式
1/2	1/2
2/4	1/2
1 000 000/33 000	1000/33
-3/5	-3/5

我们预计极少程序会在本来期望项的位置解析出有理数。注意，对于出现在算术表达式中的有理数，唯一差异是求值从运行时移动到了编译时。工具 `list_rationals/0` 可用于检查已载入程序是否在子句中包含有理数，从而可能受到兼容性影响。如果本意是写项，可以写成 `/(1,2)`、`(1)/2`、`1/ 2` 或类似变体。

- **compatibility**：以类似 `1r3` 的形式读写有理数。换句话说，它遵循上面 `natural` 的同样规则，但使用 `r` 而不是 `/`。注意，这可能与传统 Prolog 冲突，因为 `r` 可以被定义为中缀运算符。ISO 标准中作为数字语法一部分的 `0x23` 等写法也有同样问题。

有理数语法由标志 `rational_syntax` 控制；整数除法和乘方的行为由标志 `prefer_rationals` 控制。有理数算术见相关章节。

NaN、Infinity 浮点数及其语法

SWI-Prolog 支持按 Joachim Schimpf 提出的、ECLiPSe Prolog 中可用的 Prolog 标准核心浮点算术更新提案，读取和打印“特殊”浮点值。具体来说：

- **Infinity** 打印为 `1.0Inf` 或 `-1.0Inf`。任何匹配正则表达式 `[+-]?[sd+].[ ]\sd+Inf` 的序列都会映射为正无穷或负无穷。
- **NaN** (Not a Number, 非数) 打印为 `1.xxxNaN`，其中 `1.xxx` 是把指数替换为 1 后的浮点数。这些数可以被读取，并产生同一个 NaN。NaN 常量也可以用函数 `nan/0` 生成，例如：

```
?- A is nan.  
A = 1.5NaN.
```

默认情况下，SWI-Prolog 算术遵循 ISO 标准：浮点运算要么产生普通浮点数，要么抛出异常。IEEE 浮点相关章节描述了可用于支持 IEEE 特殊浮点值的 Prolog 标志。创建、读取和写入这些值的能力，有助于和支持完整 IEEE 双精度范围的语言交换数据。

强制只有下划线引入变量

按照 ISO 标准和大多数 Prolog 系统，以大写字母或下划线开头的标识符都是变量。过去，Prolog by BIM 提供过一种替代语法：只有下划线 (`_`) 引入变量。从 SWI-Prolog 7.3.27 开始，SWI-Prolog 支持这种替代语法，由 Prolog 标志 `var_prefix` 控制。与 `character_escapes` 标志一样，该标志按模块维护；默认值为 `false`，即支持标准语法。

如果代码包含大小写敏感外部语言的标识符，那么只有下划线引入变量会特别有用。例如 RDF 库中，代码常常指定属性名和类名；又如 R 接口需要指定以大写字符开头的函数或变量。词汇数据库中也常有部分项以大写字符开头，使用该选项能提高这类代码的可读性。

Unicode Prolog 源码

ISO 标准用 ASCII 字符规定 Prolog 语法。由于 SWI-Prolog 支持在源文件中使用 Unicode，我们必须扩展语法。本节描述这对源文件的影响；编写国际化源文件则在相关章节中描述。

SWI-Prolog 的 Unicode 字符分类遵循只读 Prolog 标志 `unicode_syntax_version` 报告的 Unicode 版本。注意，`char_type/2` 及相关谓词用于处理任意文本而非 Prolog 源码，它们基于 C 库的区域设置分类例程；`library(unicode)` 中的谓词会报告随附 `utf8proc` 数据的版本，该版本可能不同于语法分类器的版本（见 `unicode_version/1`）。

- **带引号的原子和字符串：**任何文字系统中的任何字符都可以用于带引号的原子和字符串。转义序列 `\uXXXX` 和 `\UXXXXXXXX`（见“字符转义语法”）被引入，用于在 ASCII 文件中指定 Unicode 码点。
- **原子和变量：**二者关系密切，因此合在一起说明。Unicode 标准为计算机语言中的标识符定义了一种语法（见 <http://www.unicode.org/reports/tr31/>）。SWI-Prolog 使用 `XID_Start` 和 `XID_Continue` 集合：标识符由一个 `XID_Start` 码点开头，后接一串 `XID_Continue` 码点。作为配置扩展，上标数字（²、³、¹，以及 ⁰⁻⁹，即 U+00B2、U+00B3、U+00B9、U+2070、U+2074..U+2079）和下标数字（₀₋₉，U+2080..U+2089）也被接受为 `XID_Continue`，从而允许 `x2` 和 `x1` 这样的变量。这类序列作为一个 token 处理。只有当 token 以下划线（`_`）开头，或以 Unicode 通用类别 `Lu`（大写字母）中的码点开头时，它才是变量；否则它是原子。注意，标题式大写字母（通用类别 `Lt`，例如 `Dž`）会开启原子，而不是变量；这不同于较早版本使用更宽泛派生属性 `Uppercase` 的行为。许多语言没有字符大小写概念；在这类语言中，变量必须写成 `_name` 这样的形式。
- **数字：**在源码中（`read_term/2`），数字字面量只使用 ASCII 数字 0 到 9。通过 `atom_number/2`、`number_codes/2` 和 `number_chars/2` 转换时，对于整数、有理数（见“有理数语法”）和浮点数，还额外接受任意 Unicode `Nd` 数字块；同一个数字中的所有数字必须来自同一个块，也就是说，如果有理数的分子使用印度文字，分母也必须如此。符号、有理数分隔符、浮点数的 `.` 和浮点指数始终是 ASCII。
- **空白：**布局字符恰好是 UAX #31 定义的 Unicode `Pattern_White_Space` 集合：U+0009..U+000D、U+0020、U+0085、U+200E、U+200F、U+2028 和 U+2029。NBSP（U+00A0）有意被排除在 `Pattern_White_Space` 之外；如果它出现在带引号材料之外，会抛出杂散字符语法错误。从文字处理器粘贴的程序偶尔会在错误位置带入 NBSP，明确报告它比静默当作分隔符更好。
- **行终止：**`Pattern_White_Space` 中有七个码点会结束一行：U+000A（LF）、U+000B（VT）、U+000C（FF）、U+000D（CR）、U+0085（NEL）、U+2028（LINE SEPARATOR）和 U+2029（PARAGRAPH SEPARATOR）。它们会终止 % 行注释，推动源码位置的行计数器，并在带引号字符串中作为反斜杠-换行续行的换行符（\ 后接行尾，再后接零个或多个空白会被消耗）。同一集合通过 `code_type/2` 和 `char_type/2` 中的 `prolog_end_of_line` 暴露给用户代码；不带前缀的 `end_of_line` 仍限制为四个 ISO/POSIX 控制码（LF、VT、FF、CR），而 11 个成员的 `Pattern_White_Space` 集合本身是 `prolog_layout`。
- **源文本中的杂散字符：**在 token 起始位置（也就是允许布局字符的位置），如果某个码点不属于任何已识别语法类别，即布局（见上文）、十进制数字、标识符起始、标识符延续、单独字符、括号开符号或引号开符号，则抛出 `syntax_error(illegal_character)`。这包括 `C0`

和 C1 控制范围、未分配码点和非字符码点、代理码点、不在 Pattern_White_Space 中的 Zs / Zl / Zp 分隔符类别 (NBSP、OGHAM SPACE MARK、NARROW NO-BREAK SPACE、IDEOGRAPHIC SPACE 等)、既不在 Pattern_White_Space 中也不在 Other_ID_Continue 中的 Cf 格式字符 (SOFT HYPHEN、ZERO WIDTH SPACE 等)、包围组合标记 (Me)，以及不属于显式上标或下标数字配置的其他数字字符 (No，例如常用分数和罗马数字形式 U+00BC..U+00BE)。

非间距组合标记 (Mn、Mc) 同样会在 token 起始位置被拒绝，因为它们不能开启标识符；但它们属于 XID_Continue，因此会并入前面的标识符 (U+0061 后接 U+0300 COMBINING GRAVE 会读作一个由两个码点组成的单 token 标识符)。

- **带引号材料内部**：在单引号原子 ('...')、双引号字符串 ("...")、反引号文本 (`...`)、Unicode 引号对 (见上文)、% 注释以及 /* ... */ 注释内部，**任何** Unicode 标量值 (U+0000 到 U+10FFFF，不包括 UTF-8 无法编码的代理码点) 都会被逐字接受。转义序列 \uXXXX 和 \UXXXXXXXX (见“字符转义语法”) 可用于可移植性和显式清晰性，而不是作为准入门槛。唯一例外是双向文本覆盖/隔离范围 (U+202A..U+202E 和 U+2066..U+2069)，它们会作为 Trojan-source 防御被拒绝 (见 unicode_atoms)。对包含控制码点或零宽码点的原子或字符串进行带引号写出 (例如 writeq/1) 时，该原子或字符串会被加引号，问题码点会用转义序列写出；见“字符转义语法”。
- **其他字符**：前 128 个字符遵循 ISO Prolog 标准。特别是，ASCII 符号字符会粘连成复合原子，产生熟悉的运算符 token，如 ==、=.、:- 等。ASCII 之外，所有 Unicode 符号字符 (通用类别 Sm、Sc、Sk、So) 以及连接符、短横线和其他标点类别 (Pc、Pd、Po) 都被视为单独字符 (solo)：每个字符各自形成一个原子，不会和相邻符号粘连。这是相对于早期版本的有意改变；早期版本中 Unicode 符号会像 ASCII 符号一样粘连成复合原子。该改变确保 ≤、€、· 等字符保持逐字符意义。由 Unicode 符号构成的运算符必须用 op/3 显式声明。其他类型的数字字符 (通用类别 No，例如分数和带圈数字) 不属于标识符集合；只有显式列出的上标和下标数字扩展了标识符。
- **括号 (成对分隔符)**：开标点/闭标点类别 Ps 和 Pe 形成括号对 (bracket pairs)：一个开字符后接一个 Prolog 项，再后接匹配的闭字符，会被读作一元复合项，其函子是两个分隔符字符拼接而成。这与 {Term} 变成 '{'}(Term) 的形态相同，只是推广到了完整 Unicode Ps/Pe 集合 (64 对，来源于按通用类别过滤的 Unicode BidiMirroring.txt)。括号内的运算符会被遵守；嵌套也按预期工作。闭字符不匹配或游离闭字符会抛出 syntax_error。与 {} 的类比是完整的：一个空对 (可只包含布局) 会读作双字符原子，而不是复合项；该原子后接 (时是函子；输出时，'<open><close>'(X) 会写作 <open>X<close>，裸原子也不加引号，二者都受写选项 brace_terms(true) 约束。
- **引号 (成对字面文本分隔符)**：起始/结束引号类别 Pi 和 Pf 形成引号对 (quote pairs)：一个开字符后接字面文本，再后接匹配的闭字符，会被读作一元复合项，其函子是两个分隔符字符拼接而成，参数则是其中的文本，形式由 double_quotes 选择 (默认为字符串，也可以是原子、codes 或 chars)。其中的文本**不会**按 Prolog 项解析；转义序列 (\n、\uXXXX 等) 会像 ASCII 带引号字符串一样处理。例如，在 double_quotes 设置为 string 时，源码文本 «hello, world» 会读作一个复合项，其函子是双字符原子 «», 唯一参数是字符串 "hello, world"。引号对来自 BidiMirroring.txt 中的 Pi/Pf 条目 (8 对)，再加上标准左右弯引号对 U+2018/U+2019 和 U+201C/U+201D；后两对不在 BidiMirroring.txt 中。闭字符不匹配和开字符无匹配都会抛出 syntax_error。

上述特性让源文本可以不经转义地包含 Unicode：

```
p(X0, X) :-          % Superscript variable profile for
    q(X0, X1),      % threaded variables.
    r(X1, X).

?- atom_number('३३३', N).    % Devanagari Nd via atom_number/2
N = 123.

?- atom_codes(≤, Cs).        % Unicode symbol stays solo
Cs = [8804].

?- term_string(T, "{a, b}"), % bracket pair (Ps/Pe)
    display(T).
{}('','(a,b))
T = {a, b}.

?- term_string(T, "{ }").    % empty pair is the atom
T = {}.

?- term_string(T, "«hello"). % quote pair (Pi/Pf)
T = '«»'("hello").
```

单例变量检查

单例变量（singleton variable）是在一个子句中只出现一次的变量。它总是可以替换为匿名变量 `_`。不过，在某些情况下，人们更愿意给这个变量一个名字。由于变量拼写错误是常见错误，如果某个变量只使用一次，Prolog 系统一般会给出警告（由 `style_check/1` 控制）。如果变量本来就应当只出现一次，可以通过让它以 `_` 开头来通知系统，例如 `_Name`。请注意，除单独的 `_` 之外，任何变量都会与同名变量共享。项 `t(_X, _X)` 等价于 `t(X, X)`，这与 `t(_, _)` 不同。

由于许多语言中的变量都必须以下划线开头，这套方案已经被扩展。首先定义两类命名变量。

- **命名单例变量**：命名单例以双下划线（`__`）开头，或以单下划线后接大写字母开头，例如 `__var` 或 `_Var`。
- **普通变量**：所有其他变量都是“普通”变量。注意，这使 `_var` 成为普通变量。一些 Prolog 方言会这样书写变量。

任何在子句中只出现一次的普通变量，以及任何出现多于一次的命名单例变量，都会被报告。下面是一些示例，右列给出警告。单例消息可用 `style_check/1` 指令抑制。

最后，名为 `<digit>` 的变量永远不会接受风格检查。这些变量名由 `write/1` 等谓词生成。这个例外也可用于把单例传给 `debug/3`。把单例传给 `debug/3` 在其他情况下会有问题：使用普通变量时，如果优化移除了 `debug/3` 语句，就会产生单例警告；使用命名匿名变量时，又会产生多例（multiton）警告。例如：

```
p(X) :-
    q(X, _0Y),
    debug(demo, 'q/2 says ~p', [_0Y]).
```

示例	警告
----	----

<code>test(_).</code>	
<code>test(_a).</code>	Singleton variables: <code>[_a]</code>
<code>test(A).</code>	Singleton variables: <code>[A]</code>
<code>test(_12).</code>	
<code>test(_A).</code>	
<code>test(__a).</code>	
<code>test(_, _).</code>	
<code>test(_a, _a).</code>	
<code>test(__a, __a).</code>	Singleton-marked variables appearing more than once: <code>[__a]</code>
<code>test(_A, _A).</code>	Singleton-marked variables appearing more than once: <code>[_A]</code>
<code>test(A, A).</code>	

语义单例变量

从 6.5.1 版开始，SWI-Prolog 区分语法单例变量（syntactic singletons）和语义单例变量（semantic singletons）。前者由 `read_clause/3` 检查，也可由带有选项 `singletons(warning)` 的 `read_term/3` 检查。后者由编译器针对在某个分支中单独出现的变量生成。例如，在下面的代码中，变量 `x` 不是**语法**单例变量，但变量 `x` 不传递任何绑定，把 `x` 替换为 `_` 不会改变语义。

```
test :-
    (   test_1(X)
    ;   test_2(X)
    ).
```

有理树（循环项）

SWI-Prolog 支持有理树（rational trees），也称为循环项（cyclic terms）。这里的“支持”定义为：面对有理树时，大多数相关内置谓词都能终止。SWI-Prolog 几乎所有内置项操作谓词，都能以与该项在栈上表示所用内存量成线性关系的时间处理项。下面这些谓词可以安全处理有理树：

`=../2`、`==/2`、`=@=/2`、`=/2`、`@</2`、`@=</2`、`@>=/2`、`@>/2`、`\==/2`、`\=@=/2`、`\=/2`、
`acyclic_term/1`、`bagof/3`、`compare/3`、`copy_term/2`、`cyclic_term/1`、`dif/2`、
`duplicate_term/2`、`findall/3`、`ground/1`、`term_hash/2`、`numbervars/3`、`numbervars/4`、
`recorda/3`、`recordz/3`、`setof/3`、`subsumes_term/2`、`term_variables/2`、`throw/1`、
`unify_with_occurs_check/2`、`unifiable/3`、`when/2`、`write/1`（以及相关谓词）。

此外，一些内置谓词会识别有理树并抛出合适的异常。算术求值属于这一类。编译器（`asserta/1` 等）也会抛出异常。未来版本可能支持有理树。能够对有理树提供有意义处理的谓词会抛出 `representation_error`。对于没有有意义解释的有理树，相关谓词会抛出 `type_error`。例如：

```
1 ?- A = f(A), asserta(a(A)).
ERROR: asserta/1: Cannot represent due to `cyclic_term'
2 ?- A = 1+A, B is A.
ERROR: is/2: Type error: `expression' expected, found
        `@(S_1,[S_1=1+S_1])' (cyclic term)
```


即时子句索引

SWI-Prolog 对多个参数提供“即时” (just-in-time) 索引。“即时”表示子句索引不是由编译器 (或动态谓词的 `asserta/1`) 预先构建的,而是在第一次调用某个可能受益于索引的谓词时构建,也就是至少有一个参数已经实例化的调用。本节描述索引逻辑使用的规则。注意,这套逻辑并不是一成不变的。系统的索引能力会继续变化。虽然这不可避免地会让某些特定用例发生回退,我们仍会尽力避免显著变慢。

下面的列表描述不同谓词和调用的子句选择过程。各个备选方案按列出的顺序考虑。

• 专用代码

编译器当前识别两种特殊情况:只有一个子句的静态代码;以及有两个子句的静态代码,其中一个子句的某个参数是空列表 (`[]`),另一个子句的同一参数是非空列表 (`[_|_]`)。注意,如果这个参数是输出参数,语义会保持不变,但效率会稍微下降。可以用 `mode/1` 把该参数声明为 `-` 来避免这种变慢。9.3.18 及以前版本只考虑第一个参数。

• 在主索引参数上线性扫描

主子句列表维护一个键 (key),通常针对第一个参数。索引键要么是常量,要么是函子 (name/arity 引用)。如果调用的主索引参数已实例化,并且子句少于 10 个,系统会使用该索引键做线性扫描,寻找可能匹配的子句。如果结果是确定性的,就使用该结果;否则系统会寻找更好的索引。7.7.2 及以前版本即使结果非确定也会使用。主索引参数是第一个满足如下条件的参数:至少有一个子句在该参数上具有可索引 (非变量) 值。9.3.18 及以前版本中,主索引固定为第一个参数。

• 哈希查找

如果上述方案都不适用,系统会考虑已有哈希表中那些对应参数已经实例化的表。如果找到特征可接受的表,就使用它。否则,系统会评估所有已实例化参数上的子句,并选择最好的候选参数来创建新的哈希表。如果没有任何单个参数能提供可接受的哈希质量,系统会搜索参数组合。最后一步是在 SWI-Prolog 7.5.8 中加入的。索引候选搜索只会在前 254 个参数上进行。

如果单参数索引包含多个具有相同 name/arity 且至少有一个非变量参数的复合项,就会创建列表索引 (list index)。后续查询中,如果这个参数绑定为复合项,JITI 索引会递归地应用到该项的参数上。这称为深层索引 (deep indexing)。深层索引是在 7.7.4 版中加入的。另见“深层索引”。

如果某个子句在本可索引的参数位置上是变量,它必须链接到所有哈希桶中。目前,如果某个谓词在特定参数上有超过 10% 的这类子句,该参数就不会被考虑用于索引。

忽略变量后,一个参数是否适合哈希,用“唯一可索引值的数量”除以“每个值的重复数量的标准差加一”来表示。早期版本只使用唯一值数量;但值分布不佳会让表不那么适合索引。这个问题由 Fabien Noth 和 Günter Kniesel 分析过。

对动态谓词而言,如果子句数量相对索引创建时翻倍,或减少到低于原来的四分之一,索引会被删除。JIT 方法会在下一次调用时重新创建合适的索引。正在运行的谓词索引不能被删除。它们会被加入该谓词关联的“已移除索引列表”。谓词中过期的索引由 `garbage_collect_clauses/0` 回收。子句垃圾收集器会基于时间和空间启发式规则自动调度。详情见 `garbage_collect_clauses/0`。

库 `library(prolog_jiti)` 提供 `jiti_list/0` 和 `jiti_list/1`,用于列出所有或部分已创建哈希表的特征。

动态谓词使用与静态谓词相同的规则建立索引，但从不应应用专用代码方案。此外，如果子句数量相对该谓词上次评估时翻倍，或缩小到低于四分之一，JIT 索引会被丢弃。后续调用会重新评估动态谓词的统计信息，并在适用时创建新索引。

JIT 索引由一组名称以 `ci_` 开头的 Prolog 标志控制，例如 `ci_min_speedup`。见 `current_prolog_flag/2`。

深层索引

如“即时子句索引”中介绍，深层索引会创建哈希表，用于区分共享同一 `name/arity` 复合项的子句。深层索引可以高效查找任意项。如果没有它，通常建议把项拍平，也就是把 $F(X)$ 转为事实中的两个参数：一个参数表示函子 F ，另一个参数表示参数 X 。只要每个项的元数相同，这种方式就很好用。另一种方式是使用 `term_hash/2` 或 `term_hash/4` 增加一列，保存该项的哈希值。这种方式可以处理任意元数，但要求我们知道该项是基项 (`term_hash/2`)，或者知道到多深的层级可以获得足够选择性 (`term_hash/4`)。

深层索引不要求具备这些知识，并且无论查询和项的实例化情况如何，都能带来高效查找。当前版本仍有一些限制：

- 在每个层级上，使用哪个索引的决策是独立做出的。未来版本可能会更智能。
- 深层索引只适用于单参数索引（可以是任意一个参数）。
- 目前，索引深度限制为 7 层。

注意，编译 DCG 时（见 DCG 相关章节），如果第一个体目标是字面量，它会被包含进子句头。下面给出一个语法及其普通 Prolog 表示形式。

```
det(det(a), sg) --> "a".
det(det(an), pl) --> "an".
det(det(the), _) --> "the".
```

```
?- listing(det).
det(det(a), sg, [97|A], A).
det(det(an), pl, [97, 110|A], A).
det(det(the), _, [116, 104, 101|A], A).
```

深层参数索引会为第 3 个列表参数创建索引。如果所有规则都以字面量开头，并且所有字面量的前 6 个元素都唯一，这会提供加速，并让子句选择变为确定性。注意，一旦可以做出确定性选择，或不存在两个子句具有相同 `name/arity` 组合，深层索引创建就会停止。

未来方向

- 可以扩展“特殊情况”。对于子句数量相对较少、哈希查找成本过高的静态谓词来说，这尤其有吸引力。
- 为少量静态子句之间的选择创建高效的决策图。
- 实现更好的判断机制，用于在深层索引和普通索引之间做选择。

体内代码索引

当前 SWI-Prolog 版本只考虑子句头来生成子句索引。这会导致无法检查头参数并在体内传递该参数，除非复制这个参数。考虑下面两个子句。二者在 Prolog 下语义相同。第一个版本会丢失子句索引，第二个版本会创建 `f/1` 参数的副本。二者都不理想。

```
p(X) :- X = f(I), integer(I), q(X).
p(f(I)) :- integer(I), q(f(X)).
```

从 SWI-Prolog 8.3.21 开始，凡是在体内任何其他目标之前发生、并针对头参数的合一，都会被特殊编译。实际效果是，被合一的项会移动到头部（从而提供索引），而使用该项的位置则直接使用相应参数。显式合一会被移除。反编译（`clause/2`）会反转这个过程，但不一定生成完全相同的项。重新插入的合一会按参数位置排序，而且变量总是位于 `=/2` 的左侧。因此：

```
p(X,Y) :- f(_) = Y, X = g(_), q(X,Y).
```

会被反编译为下面这个等价子句。

```
p(X,Y) :- X = g(_), Y = f(_), q(X,Y).
```

补充说明：

- 该转换只对**静态**代码执行。
- 合一必须在一个合取中**紧跟**在头部之后。
- 唯一例外是会跳过对 `true/0` 的调用。这允许 `goal_expansion/2` 把目标转换为 `true`，同时保留这个优化。
- 如果头参数没有被使用，体内合一仍会移动到头部。在这种情况下，反编译器不会反转该过程。因此，`p(X) :- X = a.` 与 `p(a).` 完全等价。
- 目前，无论 Prolog 标志 `optimise` 如何设置，都会启用该优化。由于这个优化会妨碍源码级调试，这一点可能并不理想。另一方面，该优化会影响确定性，而我们并不希望确定性取决于是否启用优化。

索引与可移植性

Prolog 实现的基线功能是在第一个参数上按常量和函子（`name/arity`）建立索引。如果程序需要广泛可移植性，你必须以此为假设。通常可以通过使用 `term_hash/2` 或 `term_hash/4`，并且/或者维护同一谓词的多个副本来实现：这些副本使用重新排序的参数，并配合包装器更新所有实现（`assert/retract`），以及选择合适的实现（查询）。

YAP 提供完整的 JIT 索引，包括对复合项参数建立索引。YAP 的索引机制是增强 SWI-Prolog 索引能力的灵感来源。

宽字符支持

SWI-Prolog 使用 Unicode 表示字符。Unicode 定义了范围为 $0..0x10FFFF$ 的码点 (code points)。这些码点几乎可以表示任何语言中的任何字符。此外，Unicode 标准还定义了字符类别 (字母、数字、标点等)、大小写转换以及更多内容。Unicode 是 ISO 8859-1 (ISO Latin-1) 的超集，而 ISO Latin-1 又是 US-ASCII 的超集。

SWI-Prolog 对原子和字符串对象有两种表示 (见字符串相关章节)。如果文本可以放入 ISO Latin-1，就表示为 8 位字符数组。否则，文本表示为 `wchar_t` 字符数组。除 MS-Windows 外，几乎所有系统上的 `wchar_t` 都是 32 位无符号整数，因此能够表示所有 Unicode 码点。在 MS-Windows 上，`wchar_t` 是 16 位无符号整数，因此只能表示 $0..0xFFFF$ 范围内的码点。从 SWI-Prolog 8.5.14 开始，Windows 上的 `wchar_t` 会被解释为 UTF-16 字符串。UTF-16 编码使用代理对 (surrogate pairs)，把 $0x10000..0x10FFFF$ 范围内的码点表示为 $0xD800..0xDFFF$ 范围内的两个码元 (code units)。作为 Unicode 码点，这个范围是未分配的。为保持一致性，SWI-Prolog 不接受代理对范围内的整数作为有效码点，例如：

```
?- char_code(X, 0xD800).
ERROR: Type error: `character_code' expected, found `55296' (an integer)
```

内部字符表示对 Prolog 用户完全透明。不过，使用外部语言接口的用户有时需要了解这些问题，相关内容见外部语言接口章节。

当需要从文件读取字符串字符、向文件写入字符串字符，或通过外部语言接口与其他软件组件通信时，字符编码就会显现出来。本节只处理通过流进行的 I/O，包括文件 I/O 以及通过网络套接字进行的 I/O。

流上的宽字符编码

虽然内部使用 Unicode 标准唯一地编码字符，但流和文件是面向字节 (8 位) 的，而在 8 位八位组流中表示较大的 Unicode 码点有多种方式。最流行的一种，尤其是在 Web 语境中，是 UTF-8。字节 $0..127$ 直接表示对应的 US-ASCII 字符，而字节 $128..255$ 用于对 Unicode 空间中更高位置的字符进行多字节编码。特别是在 MS-Windows 上，以字节对表示的 16 位 UTF-16 标准也很流行。

Prolog I/O 流有一个名为编码 (encoding) 的属性，它指定所用编码，并影响 `get_code/2`、`put_code/2` 以及所有其他文本 I/O 谓词。

文件的默认编码来自 Prolog 标志 `encoding`。该标志由 `setlocale(LC_CTYPE, NULL)` 初始化为 `text`、`utf8` 或 `iso_latin_1` 之一。如果识别出编码名称，就使用后两者之一；否则默认使用 `text`。使用 `text` 时，转换留给 C 库的宽字符函数处理。Prolog 原生 UTF-8 模式明显快于通用的 `mbtowc()` 模式。在 MS-Windows 上，默认值无条件为 `utf8`，不受系统代码页影响，因为 UTF-8 是源文件的事实编码，而 Windows C 运行时基于 `locale` 的宽字符函数提供的 Unicode 覆盖弱于 Prolog 自己的表。

可以在 `load_files/2` 中为使用替代编码载入的 Prolog 源显式指定编码；打开文件时可以在 `open/4` 中指定；也可以在任意已打开流上用 `set_stream/2` 指定。对于 Prolog 源文件，还提供了 `encoding/1` 指令，可用于在与 US-ASCII 兼容的编码之间切换 (`ascii`、`iso_latin_1`、`utf8` 以及许多 `locale`)。编写包含非 US-ASCII 字符的 Prolog 文件见国际化源文件相关章节；语法问题见 Unicode Prolog 源码相关章节。更多信息和 Unicode 资源见 <http://www.unicode.org/>。

SWI-Prolog 当前定义并支持以下编码：

- `octet`: binary 流的默认编码。这会使流在读写时完全不做转换。
- `ascii`: 8 位字节中的 7 位编码。等价于 `iso_latin_1`，但遇到大于 127 的值时会生成错误和警告。
- `iso_latin_1`: 支持许多西方语言的 8 位编码。这会使流在读写时完全不做转换。上面是 SWI-Prolog 的原生名称。该编码也可以使用官方 IANA 名称 `ISO-8859-1` 指定。
- `text`: 文本文件的 C 库默认 locale 编码。文件使用 C 库函数 `mbtowc()` 和 `wcrtomb()` 读写。它可能与其他某种编码相同，尤其是在西方语言环境中可能等同于 `iso_latin_1`，在 UTF-8 环境中可能等同于 `utf8`。
- `utf8`: 完整 Unicode 的多字节编码，与 `ascii` 兼容。见上文。上面是 SWI-Prolog 的原生名称。该编码也可以使用官方 IANA 名称 `UTF-8` 指定。
- `utf16be` / `utf16le`: UTF-16 编码。按字节对读取输入。`utf16be` 是大端 (Big Endian)，高有效字节在前；`utf16le` 是小端 (Little Endian)，高有效字节在后。UTF-16 可以通过代理对表示完整 Unicode。上面是 SWI-Prolog 的原生名称。这些编码也可以使用官方 IANA 名称 `UTF-16BE` 和 `UTF-16LE` 指定。为了向后兼容，也支持 `unicode_be` 和 `unicode_le`。

注意，并非所有编码都能表示所有字符。这意味着向流写入文本时，可能因为该流无法表示这些字符而出错。流在遇到这些错误时的行为可以用 `set_stream/2` 控制。初始状态下，终端流会使用 Prolog 转义序列写出这些字符，而其他流会生成 I/O 异常。

BOM: 字节顺序标记

读完“流上的宽字符编码”后，你可能已经感觉到文本文件很复杂。本节处理一个相关主题：它通常让用户的生活更轻松，但也给程序员带来另一层顾虑。**BOM**，即字节顺序标记 (Byte Order Marker)，是一种识别 Unicode 文本文件及其所用编码的技术。这类文件以 Unicode 字符 `0xFEFF` 开头；这是一个不换行、零宽的空格字符。它是一个相当独特的序列，不太可能出现在非 Unicode 文件开头，并能唯一地区分各种 Unicode 文件格式。由于它是零宽空白，甚至不会产生任何输出。这似乎解决了所有问题，或者说.....

有些格式以 US-ASCII 开始，并可能包含某种编码标记来切换到 UTF-8，例如 XML 头中的 `encoding="UTF-8"`。这类格式通常明确禁止使用 UTF-8 BOM。另一些情况下，文件中还有额外信息揭示编码，使 BOM 变得多余，甚至非法。

BOM 由 SWI-Prolog 的 `open/4` 谓词处理。默认情况下，读取文本文件时会探测 BOM。如果找到 BOM，就相应设置编码，并且可通过 `stream_property/2` 获得属性 `bom(true)`。以写入方式打开文件时，可以通过 `open/4` 的选项 `bom(true)` 请求写入 BOM。

系统限制

内存区域限制

SWI-Prolog 引擎使用三种栈。局部栈（local stack，也称环境栈 environment stack）保存用于调用谓词的环境帧以及选择点。全局栈（global stack，也称堆 heap）包含项、浮点数、字符串和大整数。最后，轨迹栈（trail stack）记录变量绑定和赋值，以支持回溯（backtracking）。除可用内存外，栈大小没有硬限制。9.3.6 以前的旧版本在 32 位硬件上每个栈有 128Mb 的硬限制。

组合后的栈大小（按线程计）有一个软限制，由可写标志 `stack_limit` 或命令行选项 `--stack-limit` 实现。目前默认限制是 1Gb。考虑到可移植性，需要修改默认限制的应用建议使用 Prolog 标志 `stack_limito`。

区域名称	描述
局部栈	局部栈用于存储过程调用的执行环境。环境占用的空间会在以下情况下回收：该环境失败；退出时没有留下选择点；备选分支被 <code>!/0</code> 谓词剪切；或自调用以来没有创建选择点并且最后一个子子句开始执行（最后调用优化）。
全局栈	全局栈用于存储 Prolog 执行期间创建的项、字符串、大整数、有理数和浮点数。该栈上的数据会在回溯到数据创建之前的某个点时回收，或在数据不再被引用时由垃圾收集回收。
轨迹栈	轨迹栈用于存储执行期间的赋值。该栈上的条目会一直存活，直到回溯到创建点之前，或垃圾收集器判定它们不再需要。由于轨迹栈和全局栈一起进行垃圾收集，过小的轨迹栈可能导致过多垃圾收集。为避免这种情况，轨迹栈会自动调整大小，使其至少达到全局栈大小的六分之一。

堆

这里所说的堆，是指 `malloc()` 及相关函数使用的内存区域。SWI-Prolog 使用这一区域存储原子、函子、谓词及其子句、记录和其他动态数据。对 `malloc()` 及相关函数返回的地址不施加限制。

其他限制

• 子句

子句唯一的限制是元数（头部参数数量），目前限制为 1024。提高这个限制很容易，而且代价相对较低；移除该限制则更困难。

• 原子和字符串

SWI-Prolog 对原子或字符串长度没有限制。原子数量没有限制。原子受垃圾收集管理。见原子垃圾收集相关章节。原子和字符串都可以表示所有 Unicode 码点，包括 0（`\u0000`）。目前，SWI-Prolog 对 ISO Latin-1 文本（码点 0..255）和包含更高码点的文本使用不同表示。后者使用 C 的 `wchar_t` 类型表示。在大多数系统上，这意味着 UCS-4，也就是 32 位无符号整数。在 Windows 上，`wchar_t` 使用 UTF-16，这意味着它不能把为代理对保留的码点表示为单个码点。未来版本可能会切换为全程使用 UTF-8。

• 项的嵌套

大多数处理 Prolog 项的内置谓词都会创建一个显式管理的栈，并针对处理项的最后一个参数进行优化。这意味着它们可以在 C 栈使用量恒定且较低的情况下处理深度嵌套的项；如果无法再分配栈，系统会抛出资源错误。目前只有 `read/1` 和 `write/1`（以及它们的所有变体）仍使用 C 栈，可能导致系统以不受控方式崩溃，也就是不会映射为可捕获的 Prolog 异常。

• 整数

SWI-Prolog 有两种整数表示。有标签整数（tagged integers）目前限制为 57 位。9.3.6 以前，32 位系统上的有标签整数为 25 位，并且还有第三种 64 位整数表示。无界整数默认由 GNU GMP 库提供。也可以由随附的 LibBf 库提供。系统也可以构建为不支持无界整数。

• 浮点数

浮点数表示为 C 原生双精度浮点数，在大多数机器上是 64 位 IEEE。

保留名称

引导编译器（见 `-b` 选项）不支持模块系统。由于系统的大部分内容本身就是用 Prolog 编写的，我们需要某种方式避免与用户的谓词、数据库键等发生名称冲突。与 Edinburgh C-Prolog 一样，所有应对用户隐藏的谓词、数据库键等，都以美元符号（\$）开头。

SWI-Prolog 和 32 位机器

如今的大多数平台原生就是 64 位，并且更推荐使用 64 位应用。当前版本的 SWI-Prolog 主要面向 64 位平台。32 位平台仍受支持，因为它们仍用于嵌入式设备，并且 WASM（Web Assembly，见 WASM 版本相关章节）对 64 位的支持仍然较弱。

虽然当前稳定的 9.2 系列仍有真正的 32 位版本，其中 Prolog 数据结构基于 32 位字（word）单元；但 9.3 开发系列会把所有 Prolog 数据都表示为 64 位单元，而不管硬件指针大小如何。这提供了更好的统一性，并避免了 32 位 9.2 系列的 128Mb 栈限制。

数据表示中的一些选择，例如标签（tag）位的位置和数量，仍然基于 32 位单元。预计这会在适当时候改变，从而简化代码并提升性能。

二进制兼容性

SWI-Prolog 首先尝试维护版本之间的源代码兼容性。数据和程序通常也可以用二进制形式表示。这涉及多个接口，而这些接口具有不同程度的兼容性。相关版本号和签名可通过 `PL_version_info()`、`--abi-version` 以及 Prolog 标志 `abi_version` 获得。

• 外部扩展

可动态载入的外部扩展通常依赖体系结构、C 编译器的 ABI 模型、动态链接库格式等。它们还依赖 `libswipl` 提供的 `PL_*` API 函数的向后兼容性。

兼容的 API 允许以二进制形式分发外部扩展，尤其适用于编译较复杂的平台（例如 Windows）。因此，这种兼容性在优先级列表中很靠前，但偶尔仍必须妥协。

对应版本信息：`PL_version_info()` 中的 `PL_VERSION_FLI`；`abi_version` 键：`foreign_interface`。

• 二进制项

项可以使用 `PL_record_external()` 和 `fast_write/2` 表示为二进制格式。由于这些格式用于在数据库中存储二进制项，或在 Prolog 进程之间以二进制形式传递项，因此会非常谨慎地维护兼容性。

对应版本信息：`PL_version_info()` 中的 `PL_VERSION_REC`；`abi_version` 键：`record`。

• QLF 文件

QLF 文件（见 `qcompile/1`）是 Prolog 文件或模块的二进制表示。它们把子句表示为虚拟机（VM）指令序列。其兼容性依赖 QLF 文件格式以及 VM 的 ABI。系统会以一定程度的谨慎来维护兼容性。

对应版本信息：`PL_version_info()` 中的 `PL_VERSION_QLF`、`PL_VERSION_QLF_LOAD` 和 `PL_VERSION_VM`；`abi_version` 键：`qlf`、`qlf_min_load`、`vmi`。

• 保存状态

保存状态（见 `-c` 和 `qsave_program/2`）是一个 zip 文件，使用与 QLF 文件相同的表示包含整个 Prolog 数据库。保存状态可以包含额外资源，例如外部扩展、数据文件等。除 QLF 文件的依赖性问题外，内置谓词和核心库谓词还可能调用内部外部谓词。公共内置谓词和内部外部谓词之间的接口经常变化。稳定分支中的补丁级发布会尽可能维持兼容性。

相关 ABI 版本键与 QLF 文件相同，另加一项：`PL_version_info()` 中的 `PL_VERSION_BUILT_IN`；`abi_version` 键：`built_in`。